

Football Shirt Business

SQL Database

Computer Science NEA

Name: Frederick Zeki Springett

Candidate Number: 9097

Centre Name: Barton Peveril College

Centre Number: 58231

0 Contents

0 Contents	2
1 Analysis	4
1.1 Statement of Problem	4
1.2 Background	4
1.3 End User	4
1.4 Initial Research	5
1.4.1 Existing, similar programs	5
1.4.2 Potential Abstract Data Types / Algorithms	7
1.4.3 First Interview	7
1.4.4 Key Components	8
1.5 Further Research	9
1.5.1 Prototype	9
1.5.2 Second Interview	13
1.6 Objectives	14
1.7 Modelling explain modelling more explain entities	15
2 Design	17
2.1 High Level Overview	17
2.1.1 Choice of Programming Languages and Libraries	17
2.2 Menu System	17
2.2.1 Design of Menu System	17
2.2.2 Flowchart of Menu System	18
2.2.3 Screenshots of Menu System	19
2.2.4 Pseudo Code for the general Error Handling System for User Inputs	25
2.3 Database Design	26
2.3.1 Queries	26
2.3.2 SQL queries that use data inputted by the program	27
2.3.3 Table set up	29
2.3.4 Tables	30
2.4 Analytics and their Representation	36
2.4.1 Least Squares Regression	36
2.4.2 Pseudo Code of Least Square Regression	36
2.4.3 Product Moment Correlation Coefficient	37
2.4.4 How duplicate data is dealt with	39
2.4.5 The graphs chosen and why	40
3 Testing	41
4 Evaluation	48
4.1 Overall Effectiveness of the System	48
4.2 Evaluation of Objectives	49
4.3 End User Feedback	51

4.4 System Improvements	52
5 Technical Solution	53
Directory of Technical Skills	53
Code:	54

1 Analysis

1.1 Statement of Problem

The problem to be solved is that my friend (Finlay) owns a football shirt selling business, which is rapidly growing, and therefore Excel is becoming too slow and difficult to use for the large amount of data that he has to store. Furthermore, none of his employees know how to use Excel and he has little experience of using it himself. Therefore, he is unable to teach them how to use it for him. This means that he is wasting time on tracking data on the spreadsheet giving him less time to attend to other parts of his business. If an Excel spreadsheet is poorly made then it can lead to errors that make it even more difficult to use. Therefore he is becoming very stressed while trying to use his Excel spreadsheet and could also lose out on revenue because of mistakes being made within the spreadsheet that result in the wrong data being entered into certain fields. This can be because spreadsheets are hard and frustrating for people who have little experience with the tool and it becomes even more difficult the more data you add. Overall the main issue is that Excel spreadsheets become hard to use once you enter large amounts of data that most business owners do not have time to keep track of.

1.2 Background

Finlay's business is currently responsible for manufacturing the shirts and then working with a shipping company to send the shirts to the customer. So he has to store information for shipping such as address, country (they currently only ship to the UK), city, postcode and name. As well as shipping information, he also has to store information regarding the customer, such as first name and last name. He also has to store order information such as order ID, customer ID, order date, product ID, price, product name, and quantity. He also has many products that have various items of information about them that must be stored within the database.

Currently, his business has around 50 employees and, as mentioned, before his business is growing so it will be better to get this switch to a faster alternative sooner rather than later. His shirt's prices range from £30-£80 with the average price being around £40. His shirts are targeted at lower-end consumers due to the prices being cheaper than the franchise sportswear businesses.

1.3 End User

My program is being made for my friend Finlay Gilbert who wants to have a well-made, easy-to-use database that will keep track of all the necessary parts of his football shirt business. Finlay only wants the best for his business so I trust he will provide unbiased, constructive feedback on my program, which I will use to improve it. He is very familiar with his business and what needs to be tracked and how it should be represented to him and his employees. Therefore he will be able to provide feedback relating to the database and how it is structured. He is aware of SQL but he has little experience with programming, this is why he has been using Excel for so long.

Here is a screenshot of his current customer table which he made in excel. He clearly lacks a basic understanding of how to use excel and it is very confusing to follow.

	A	B	C	D	E	F	G
1	customer ID	First Last name	address	postage code	country	city	
2		1 Bob Shearer	14 Marlon Cresent	SO40RDX	United Kingdom	brighton	
3	2 Alan Shaw	17 Penny Lane	SO10PQX	United Kingdom	Newcastle		
4	3 Guy Walker	69 montgomery road	SO40TYB	United Kingdom	Weymouth		
5	4 Paul Blart	89 ivy lane	PO67MXV	United Kingdom	Portsmouth		
6	5 Luke Bennet	16 Dudley Hill	BL542BX	United Kingdom	Blackburn		
7	6 Brian Taylor	10 MouseHole Lane	LO412TU	United Kingdom	London		
8	7 Joe Ward	11 MouseHole Lane	LO422TU	United Kingdom	London		
9	8 Theo Warne	12 Alfred Road	BO452BG	United Kingdom	Bournemouth		
10	9 Declan Simmonds	12 Alan Road	SO128BE	United Kingdom	Southampton		
11	10 Neil Warlock	27 Happy Road	SO135RX	United Kingdom	Southampton		
12	11 John McGinn	89 Victoria Lane	LO78BRY	United Kingdom	London		
13	12 Marshal Kane	48 Stanley High Street	LO12BGT	United Kingdom	London		
14	13 Boris Banks	17 Church Road	Ma26FWA	United Kingdom	Manchester		
15	14 Fred Keane	89 Bridge Street	SO402BE	United Kingdom	Southampton		
16	15 Barry Alan	41 Victoria road	MA457BG	United Kingdom	Manchester		

1.4 Initial Research

1.4.1 Existing, Similar Programs

A similar program to the one I will be creating is Excel (the program my end user currently uses), this is a spreadsheet program used for organisation, analysis and storage of data. It is usually used for financial analysis but can be used for any type of business that needs to store data. However, it is quite basic and has limited ability to connect multiple related tables together, which is what I plan to do for my program.

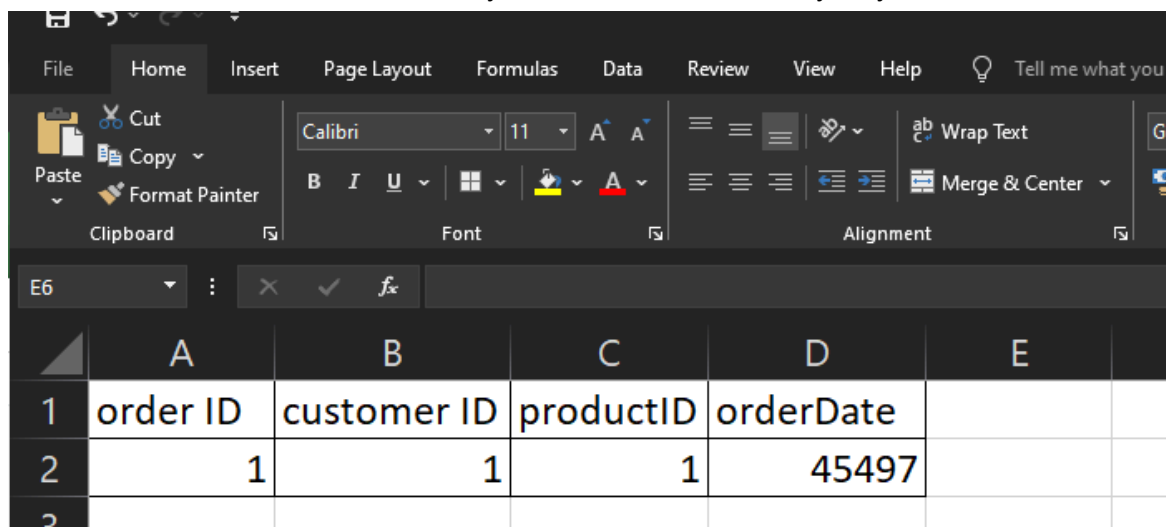
Another example of a similar program is Airtable. This is a database management system/project management program where you can make your own databases online. This software is good because it is completely online, which allows collaboration between users very easily. Also, it is very flexible and lets you add and remove columns.

Airtable and Excel are not made specifically with a football shirt-selling business in mind.

This means that specific things that apply to this unique business will not be available in Airtable or Excel. It is also hard to connect tables together in both of these programs, which is a problem for tables like orders and products because they use information from each other.

Screenshots of Excel

In these two screenshots you can see that Excel has a graphical system that allows you to add columns by writing in the boxes. In the second screenshot you can see how different tables can be accessed, however they are not connected in any way.



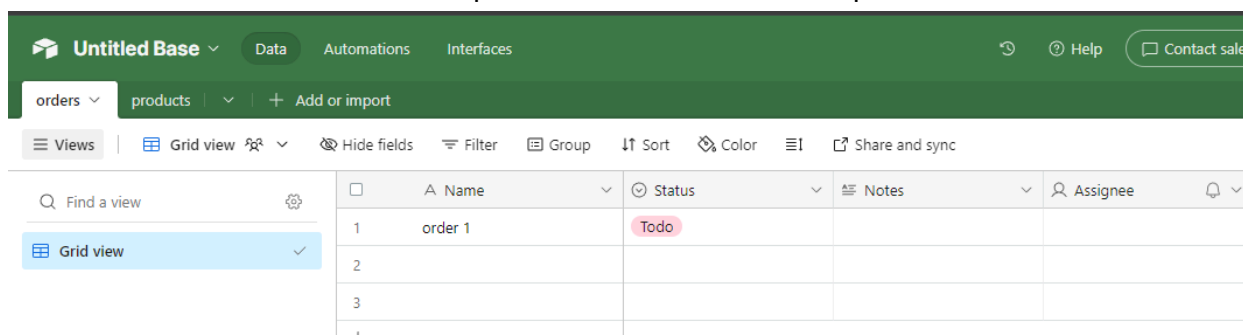
	A	B	C	D	E
1	order ID	customer ID	productID	orderDate	
2	1	1	1	45497	
3					

Here you can see the two tables in Excel.



Screenshot of Airtable

Airtable has an almost identical setup to Excel. It also has similar pros and cons.



	A Name	Status	Notes	Assignee
1	order 1	Todo		
2				
3				

Pros

I like that they both are very flexible and easy to add or remove columns.

I like how excel has statistical and mathematical functions.

I like how easy it is to create C# easily graphs using the data inputted.

cons

I don't like how these programs are not custom to our database, which means certain features will be difficult implement.

I don't like how it is difficult to connect different tables together.

I don't like how there is no way to connect it to

Features I will borrow from these programs:

- Add and remove columns easily
- Statistical functions
- Graph functions based on data inputted

Features these programs don't include that I will include:

- Make my program custom made for my end user's database
- Make it easy to connect different tables together.

1.4.2 Potential Abstract Data Types / Algorithms

I will need to use a data structure such as a dictionary to store information retrieved from an SQL query. I could also potentially use an algorithm such as least squares regression or product moment correlation coefficient (PMCC) calculation to predict future data based on what is already in the database. Least squares regression is a way of predicting future data inputs by creating a line of best fit through the current data. This would be useful for my end user because he can then predict how he will be performing in the future. Product moment correlation coefficient calculation will then check the validity of this prediction by analysing the two data sets (X and Y in the context of a graph) and seeing the correlation between the two data sets. This basically tells the user if the line of best fit calculated and the prediction made by the least squares regression is accurate. This would help the user see if the prediction made by the program is worth taking into account or not.

1.4.3 First Interview

Interviewer

Interviewee (Finlay)

What data do you store and how many tables do you have currently?

Is there anything you want to change about this?

I currently have 6 tables and store information about my business that involves the product, the order, the shipper, the customer and my employees. However, we can remove the shipper table because that information does not need to be stored anymore. Also, employees can be removed. We also do not need this anymore. However, I want you to add order details and football club details in place of the removal of these 2 tables, to make it easier to manage orders and products.

How would you like the data to be visually represented?

I would like the data to be visually easy to read so that me and all of my employees can understand it, so in a list or possibly a table that is printed to the console.

Are there any important features that you would like me to add?

I would like you to add a table that just displays some basic stats of the database such as the total number of customers, total number of products and total number of orders. Also, have a date attached to these values so that we can see how our business is growing.

Anything else you would like me to add?

Maybe a way to predict how sales and other data change in the future based on the data already in the database. Maybe you could represent this as some type of graph like a bar chart or a scatter plot graph to show the progress and trends of the business.

What do you think about the idea of printing certain data points as a different colour depending on the number of duplicates of that data point there are?

I think that is a great idea. This would mean that every data point would be represented on the graph in some way. This would make the graph much more informative. This probably means that a scatter plot would be more appropriate.

After completing this interview it is clear to me that it is important to create a fully normalised database and add 2 new columns that improve the manageability of orders. I also need to represent the data in a table in the C# console so that the end user and his employees can use it. It is also important that I add a table that displays basic stats about the database and will include stats such as total number of customers, total number of products and total number of orders. This interview has shown me that I need a way to predict potential future data inputs and identify trends. I also need to create a way to represent a scatter graph in

the console so that Finlay can see the progress of his database. Finally, I also need to show how many duplicates each data point has on the scatter graph so that each data point is represented on the graph.

1.4.4 Key Components

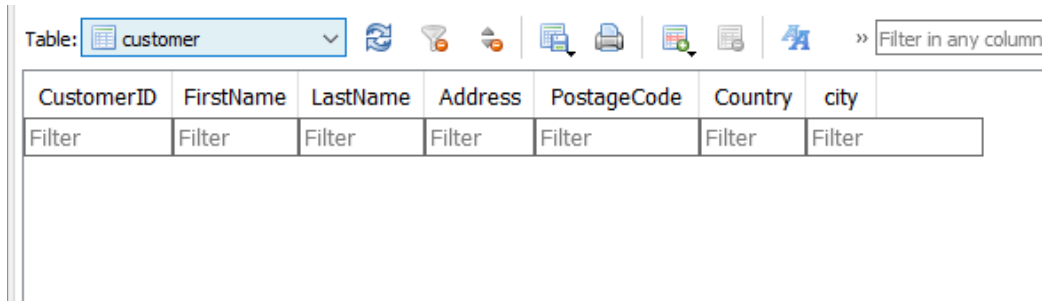
After completing my research and completing this interview I have determined that I will need to create an SQL database with the current Excel tables. In addition it will need 2 new tables that will manage the orders they will be called order and order details. Furthermore, the end user has asked me to print the tables from SQL to the C# console. I will also need to create a new table called "stats", which will hold basic statistics on the database such as total customers, products and orders. I will also need to use an equation and create an algorithm to predict future potential data inputs, this will help Finlay make decisions for his business based on predicted data inputs. This will be represented as a scatter graph in the console, which he suggested. I have chosen a scatter graph because it is relatively simple to implement into the C# console and also shows data trends over time, a bar graph does not show this as well in my opinion. The algorithms / equations I will be using to predict future inputs will be least squares regression, which will predict the future data input. I will also use the product moment correlation coefficient to show the validity of this prediction. It reflects whether the data the prediction is made off of had a trend (meaning the prediction will be more accurate) and if the end users should trust this prediction or not. For an SQL database to be as practical as possible, it must be fully normalised into 3rd normal form. Normalising a database means to make all the data in one table relate to the main key (keys are commonly represented as IDs). 3rd normal form refers to a state where the database has undergone changes to achieve a state where it is fully normalised.

1.5 Further Research

1.5.1 Prototype

Create the customer table

```
CREATE TABLE customer(CustomerID INT PRIMARY key, FirstName  
VARCHAR,LastName VARCHAR,Address VARCHAR,PostageCode  
VARCHAR,Country VARCHAR,city VARCHAR)
```



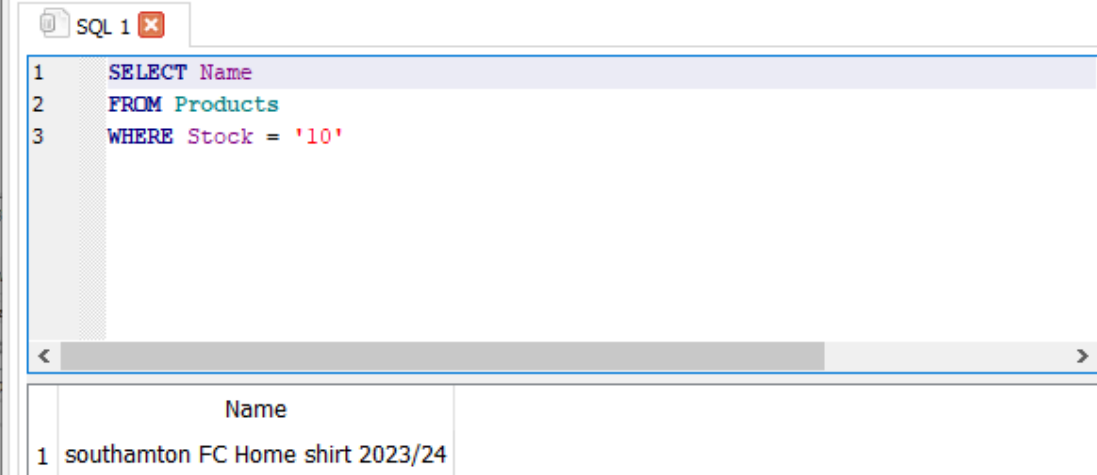
CustomerID	FirstName	LastName	Address	PostageCode	Country	city
Filter	Filter	Filter	Filter	Filter	Filter	Filter

Simple insert into query to add records to the table

```
INSERT INTO customer  
VALUES('3','Alan','Shaw','17 Penny Lane','NE10PQX','United Kingdom','Newcastle');
```

Select all the products that have 10 shirts in stock

```
SELECT Name
FROM Products
WHERE Stock = '10'
```

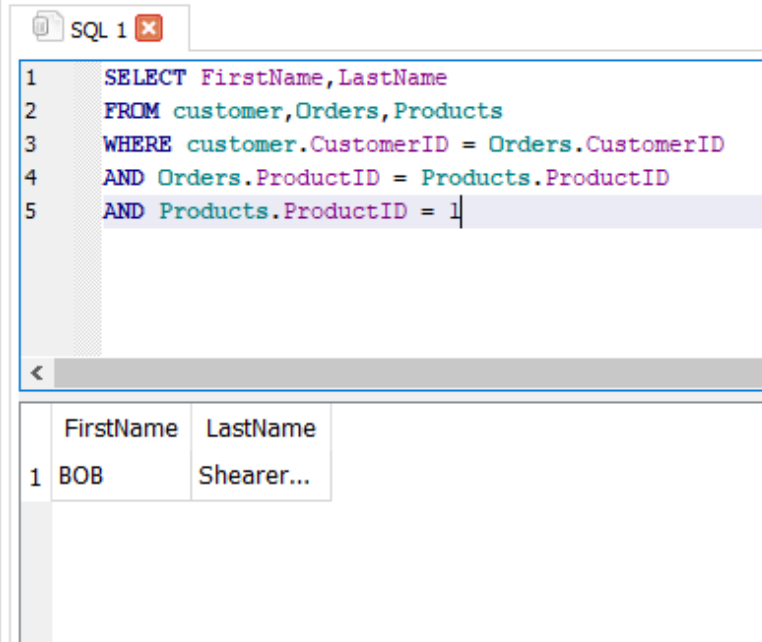


The screenshot shows a SQL query editor window titled 'SQL 1'. The query is: `SELECT Name FROM Products WHERE Stock = '10'`. Below the query editor, the results are displayed in a table with one column, 'Name'. The first row contains the text 'southampton FC Home shirt 2023/24'.

	Name
1	southampton FC Home shirt 2023/24

Select the first and last names of customers who have ordered a product with the product ID of 1

```
SELECT FirstName, LastName
FROM customer, Orders, Products
WHERE customer.CustomerID = Orders.CustomerID
AND Orders.ProductID = Products.ProductID
AND Products.ProductID = 1
```

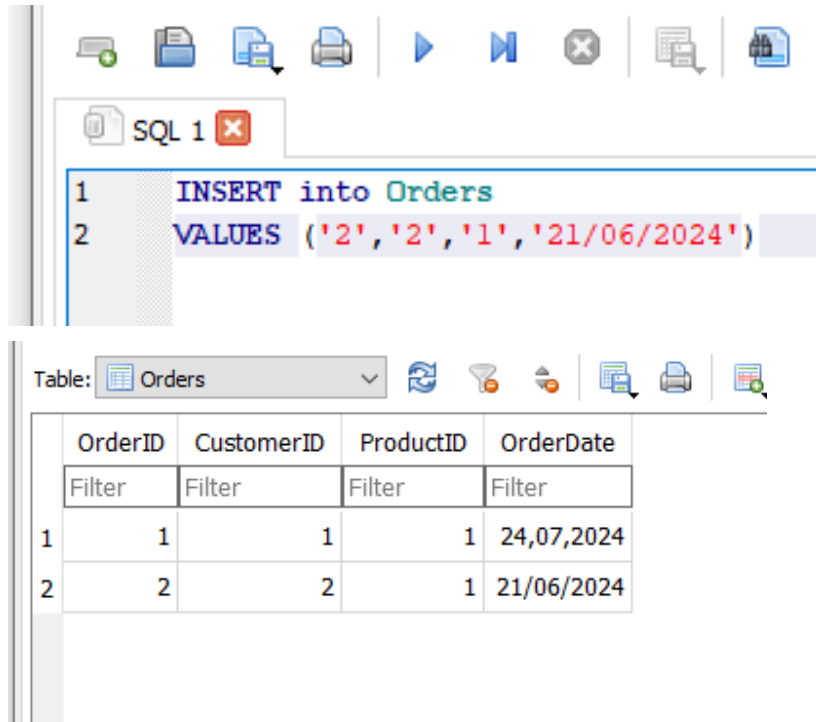


The screenshot shows a SQL query editor window titled 'SQL 1'. The query is: `SELECT FirstName, LastName FROM customer, Orders, Products WHERE customer.CustomerID = Orders.CustomerID AND Orders.ProductID = Products.ProductID AND Products.ProductID = 1`. Below the query editor, the results are displayed in a table with two columns, 'FirstName' and 'LastName'. The first row contains the values 'BOB' and 'Shearer...'.

	FirstName	LastName
1	BOB	Shearer...

Simple insert into query that adds records to orders

```
INSERT into Orders  
VALUES ('2','2','1','21/06/2024')
```



The screenshot shows a SQL IDE interface. At the top, there is a toolbar with icons for saving, running, and other database operations. Below the toolbar, a tab labeled 'SQL 1' is active, displaying the following SQL query:

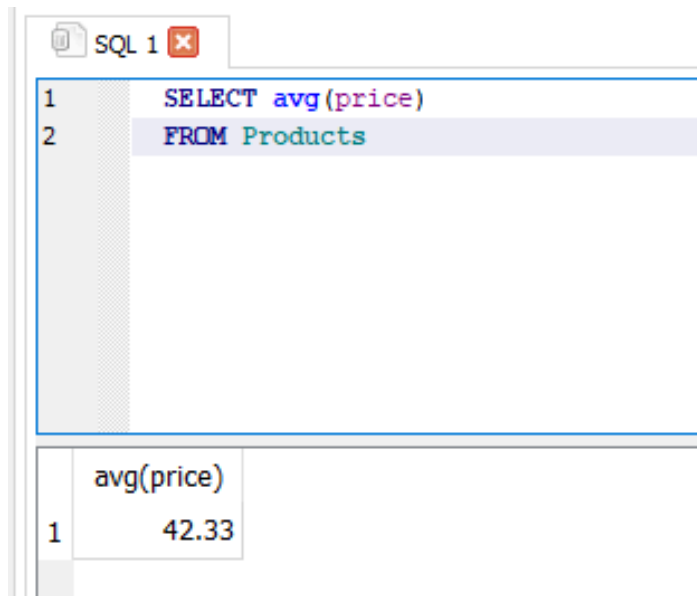
```
1 INSERT into Orders  
2 VALUES ('2','2','1','21/06/2024')
```

Below the query editor, the 'Table: Orders' dropdown is selected. The table data is displayed in a grid with columns: OrderID, CustomerID, ProductID, and OrderDate. The first row shows OrderID 1, CustomerID 1, ProductID 1, and OrderDate 24,07,2024. The second row shows OrderID 2, CustomerID 2, ProductID 1, and OrderDate 21/06/2024.

	OrderID	CustomerID	ProductID	OrderDate
1	1	1	1	24,07,2024
2	2	2	1	21/06/2024

Simple aggregate query that finds the average price of the products

```
SELECT avg(price)  
FROM Products
```



The screenshot shows a SQL IDE interface. At the top, there is a toolbar with icons for saving, running, and other database operations. Below the toolbar, a tab labeled 'SQL 1' is active, displaying the following SQL query:

```
1 SELECT avg(price)  
2 FROM Products
```

Below the query editor, the result of the query is displayed in a grid with one column: avg(price). The first row shows the value 42.33.

	avg(price)
1	42.33

Alter the table orderdetails to have a price column

```
ALTER TABLE OrderDetails
```

```
ADD price INT
```

Get the total amount of orders each customer has made

```
SELECT (FirstName || ' ' || LastName) AS 'fullname', count(Orders.CustomerID) AS
'total'
FROM customer , Orders
WHERE customer.CustomerID = Orders.CustomerID
GROUP BY Orders.CustomerID
```

```
SELECT (FirstName || ' ' || LastName) AS 'fullname', count(Orders.CustomerID) AS 'total'
FROM customer , Orders
WHERE customer.CustomerID = Orders.CustomerID
GROUP BY Orders.CustomerID
```

	fullname	total
1	BOB Shear...	2
2	Alan Shaw	1

Get the full names of the customers who have orders with prices larger than the average prices of all the shirts

```
SELECT (FirstName || ' ' || LastName) AS 'full name', price AS 'prices larger than
average', (SELECT avg(price) FROM Products) as 'average'
FROM OrderDetails, Orders, customer
WHERE OrderDetails.OrderID = Orders.OrderID
AND Orders.CustomerID = customer.CustomerID
AND price > (SELECT avg(price) from Products)
```

	full name	prices larger than average	average
1	BOB Shear...	50	49.2475
2	Alan Shaw	50	49.2475
3	Guy Walker	70	49.2475

Insert statistics into a newly made stat table using insert into with select

```
INSERT INTO stats(totalProduct, totalCustomer, totalOrder)
VALUES((SELECT COUNT(*) FROM Products),
(SELECT COUNT(*) FROM customer),
(SELECT COUNT(*) FROM Orders))
```

Table: stats

	totalProduct	totalCustomer	totalOrder
	Filter	Filter	Filter
1	3	3	4

From creating my prototype, I have learnt how to complete certain complex and simple SQL queries that are useful for knowing important information for the business. They also give me a foundation for creating similar useful queries. These complex SQL commands are very important because they are needed when using a large cross table database. This information displayed by the query can be used by the end user to make informed decisions relating to his business. It is important to know these queries so that I can add them to my program so that they can easily be used by a user who doesn't know the commands. I believe the prototype was successful and I will be using parts of these queries. Even though most of them are quite basic they are the foundations for more complex queries.

1.5.2 Second Interview

Interviewer

Interviewee

Is the prototype what you imagined and is there anything missing?

The SQL commands are similar to what I expected as long as there are many more commands similar to the ones shown in the prototype then the program will fulfil my needs.

Do you think I should add a dedicated part of the program that gets more complex stats about the database?

Yes, I think you should definitely do that because it would really help me see more intricate and specialised stats about the database that would help me with specific parts of my business such as chasing up customers who have never made orders but are on our system.

Is there any other functionality you would like me to add regarding the order tables?

I think you should also create a way to search through all the orders to find a certain one. This would allow me to for example search for all orders from a certain customer.

What data do you think the graphs should plot in my program?

I think you should add one or two that relate to the profit and sales of the business and 3 more that relate to the basic stats of the business such as the total products, customers and orders. This will help me track how our business and sales grow.

After completing this second interview I have decided on a page that will show 4 complex SQL queries that show the end user useful information about his business. These queries will be the total number of orders each customer has made, the names of the customers who have orders with prices larger than the average price of all the shirts, the products with the most sales and the name and address of each customer on the database who has not made an order. I will also create many different ways to search through all of the orders so that the end user can search for certain orders that concern a certain customer, football club, city or price. I have also made the decision to add 5 different data axis for the graphs to plot. These include The sum of profit from each day, The price of the shirt that was sold each day, The total products each day, The total number of customers each day and The total amount of orders each day.

1.6 Objectives

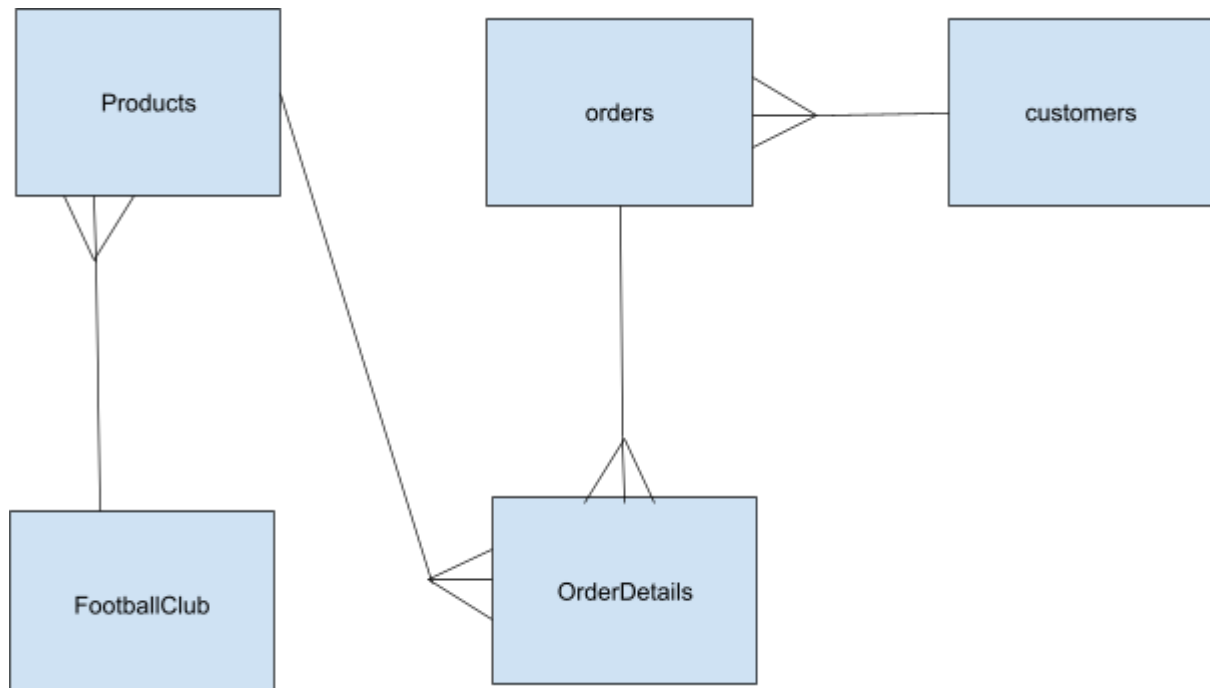
The overall aim of this program is to create an SQL database and connect it to C# so that a program can be run to make it easy for my end user to use SQL queries without having to know syntax or having to type them in. The program and database will be made with his business in mind so that it is easy for him to use. It should have a way to add orders to the database, show statistics about the database and plot data onto a scatter graph to show trends over time. This data should be analysed by a least squares regression algorithm to predict future data inputs, The data will then be analysed by a PMCC algorithm to see validity of this prediction.

- 1) Create an SQL database that stores information for the business.
- 2) Connect the SQL database to a programming language so that queries can easily be used in a program.
- 3) The database will store and display information to the user and contain 6 normalised tables:
 - a) Customer
 - b) Order
 - c) Order information
 - d) Product
 - e) Football club information
 - f) Stats
- 4) The program will check if user inputs are valid to avoid exceptions/errors
- 5) Functionality for the table such as:
 - a) Insert values into the database
 - i) Make it so that before you insert values into the table it shows you the last row of data entered.
 - b) Calculate statistics for the database and insert them in their own table for the user to see
 - c) The user will be able to alter the columns from all the tables in the database (apart from the stats table):
 - i) Add new columns
 - ii) Remove columns
 - iii) Rename columns
 - d) Add a way to edit data within the database from the program
 - e) Create a way to search through all orders to find specific records. Ways to search will include:
 - i) search for orders from a certain customer
 - ii) search for orders that include a certain team
 - iii) search for orders with a certain price
 - iv) search for orders from a certain city
- 6) Create two order tables (orders and order information) :
 - a) An order table contains the top-level order information such as the customer ordering and the date

- b) An order information table includes more details about the order itself and what it contains (product and quantity)
 - c) The tables are fully normalised
 - d) A way to represent the two order tables in a way that is useful to the end user such as an “all orders table”
 - e) Make a way to add multiple products to one order.
- 7) Create a graph in the console to represent useful data to the end user and calculate useful figures such as product moment correlation coefficient and least squares regression (the different graphs will represent each month).
- These graphs will be :
- a) The sum of profit from each day
 - b) The price of each shirt sold each day
 - c) The total products each day
 - d) The total amount of customers each day
 - e) The total amount of orders each day
- 8) Make sure any duplicate data is plotted in a different colour to non-duplicate data so that every datapoint is represented on the graph
- 9) Use complex SQL to create useful statistics that connect multiple tables together.
- These statistics will include:
- a) The total amount of orders each customer has made
 - b) The full names of the customers who have orders with prices larger than the average prices of all the shirts
 - c) The product with the most sales
 - d) The name and address of each customer on the database who has not made an order

1.7 Modelling

Entity relationship diagram



This ER diagram above shows how all the tables are connected to each other within the database. This proves the database is fully normalised.

Relationships

This is another way of representing the database and how the tables are connected. Underlined columns are primary keys and columns with a star (*) next to them are foreign keys.

customers(customerID,FirstName,LastName,Address,PostageCode,Country,City)

Products(ProductID,Name,stock,description,size,price,ClubID*)

Order(OrderID,CustomerID*,orderDate)

OrderDetails(OrderDetailsID,OrderID*,ProductID*,quantity)

FootballClub(clubID,ClubName)

The different entities

The “customers” entity (table) stores all information about the customer which includes information regarding their identity and their location. The “products” entity stores information about products and has a foreign key relating to the Football club table because each shirt will be associated with a club. It stores information describing the visual look of the shirt, the stock of the shirt and the price. The order table stores information regarding which order it is (the order ID), the customer ordering (their ID) and the data this order was made. The order details table stores more details about the order. It has a foreign key which is the order ID and a primary key order details ID. It is set up in this way because this allows orders to have multiple products stored within one order. Finally the football club which just stores a primary key of football club ID and stores the club name. This table exists to help categorise the shirts. All of the tables have a primary key which is their ID.

2 Design

2.1 High Level Overview

This program is a management system for a database that stores information for a football shirt-selling business. The program will use SQL using DB browser for the SQL database and queries and will be connected to the programming language C# by downloading the SQLite library. I have chosen these programming languages and libraries because they have great connectivity and makes it easy for the user to interact with the database through the C# program. The program is based around a large menu system where the user will choose what functionality of the program they want to access. The table has all the functionality of SQL allowing the user to manipulate almost all of the tables however they want. My program also includes a way to make orders of shirts and then add them into the fully normalised tables easily and there is also a way to view all of the orders and search through them to find specific ones. Finally, there is also a graph and statistics function that will show the user useful analytics about their database. Once they have chosen what they want to do the program will then complete the action and ask for any inputs needed to make the SQL work as intended. The program flows as a loop where after the action the user chooses has been completed, the user will be sent back to the main menu where they can use a different part of the program. The program is written in C# and SQL (using SQLite) because they have good connectivity allowing me to interact with the database and make queries from C#.

2.1.1 Choice of Programming Languages and Libraries

I have chosen C# as the programming language because it has great libraries that connect well to SQL, which is very important when making a database project. It is a language that I also feel very comfortable with. This means that I will be able to program the project to the best of my ability.

I have also chosen to use SQL for similar reasons to choosing C#. It has great connectivity with C# and it is also a language I am confident with. Another reason is that SQL has better performance than Excel, which is important because that was one of the key issues my end user was facing. I chose to use the SQLite library because it makes completing SQL queries in C# easy.

2.2 Menu System

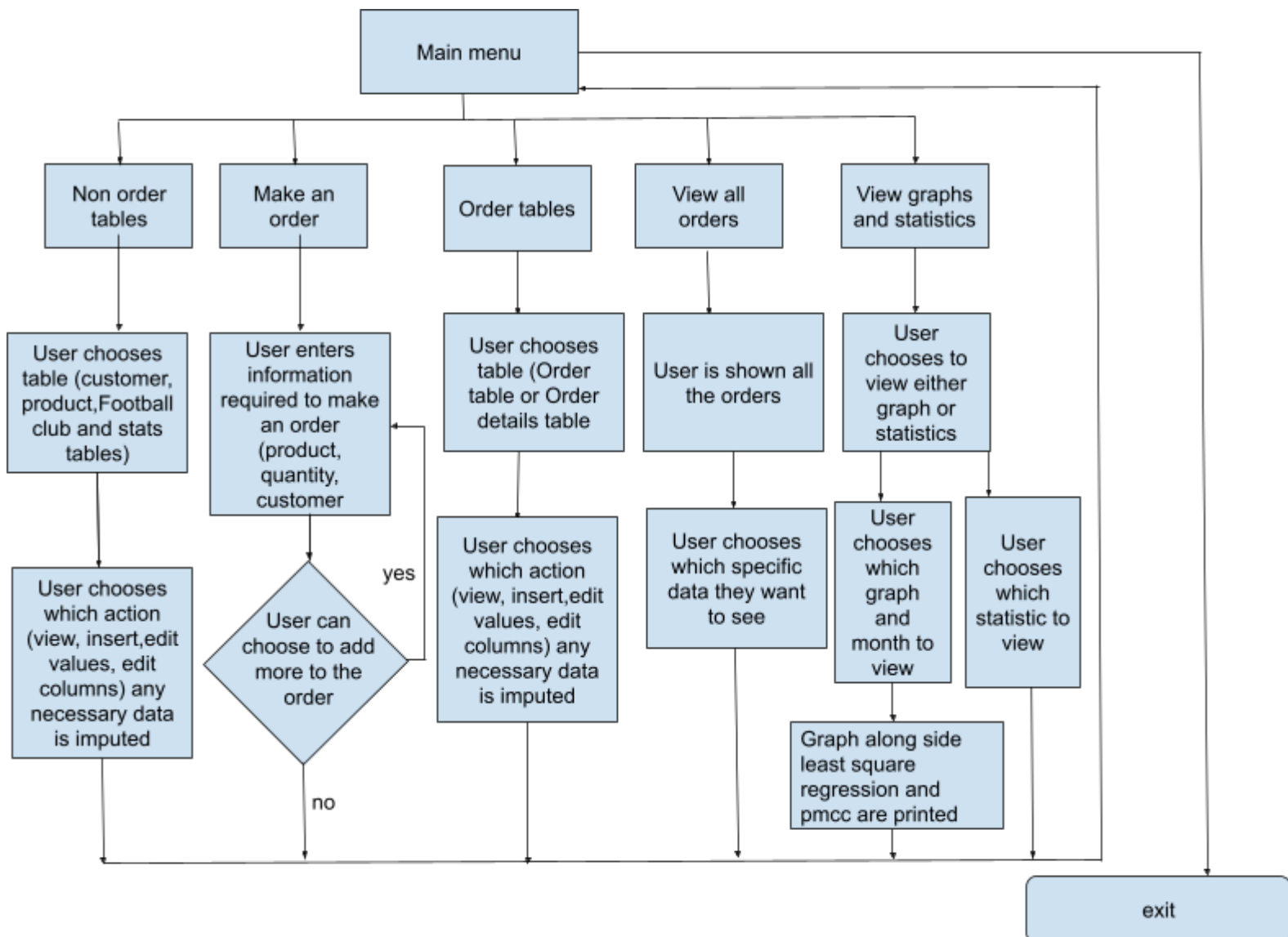
2.2.1 Design of Menu System

The basic design of the menu system works like a loop where you start in the main menu and then choose which part of the program you would like to use or which table you would like to edit/view. You then input any necessary information into the program and then the program enters the data into SQL. You are then returned to the main menu where you can complete the loop again. Every time the user is prompted to enter data into the console, such as entering a number to access a part of the menu, there must be exception handling

code where any number bigger than the available menu options that is entered is addressed by the program (without allowing the program to crash). There must also be a way for the program to handle the user entering something other than an integer (such as a string) so that the program does not crash.

2.2.2 Flowchart of Menu System

This flowchart shows the basic flow of the menu and shows how it loops and how it can be terminated.



2.2.3 Screenshots of Menu System

Non-order table UI

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTIONS\main menu.py
MAIN MENU
which action do you want to complete?
-----
1)Non-order tables
2)Make an order
3)order tables
4)View all orders
5)view graphs and statistics
9)exit
Please enter your choice:
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTIONS\main menu.py
MAIN MENU
which action do you want to complete?
-----
1)Non-order tables
2)Make an order
3)order tables
4)View all orders
5)view graphs and statistics
9)exit
Please enter your choice: 1

which table would you like to access?
1)Customer table
2)Product table
3)Football club table
4)Stats table
9)Exit
_
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTIONS\main menu.py
what action would you like to complete
1)view the table
2)insert values into the table
3)edit the records in the table
4)alter the table's columns
9)main menu
Please enter your choice: _
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Database Technical Soloution.exe
the last row of data entered:
17 test test test test test test

please enter data for each column : firstname,lastname,address,postal code, country,city (Customer ID will be automatically incremented)
enter first name:
```

Make an order UI

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Database Technical Soloution.exe
which product would you like to order

ProductID| Name| size| Price|
1| southampton FC Home shirt 2023/24| M| 50|
2| southampton FC away shirt 2023/24| M| 36|
3| Arsenal FC home shirt 2023/24| L| 40|
4| Real Madrid FC home shirt 2023/24| S| 70|
5| Poland national team home shirt 2023/24 | L| 45|
6| Argentina national team home shirt 2023/24 | M| 60|
7| Man city home shirt 2023/24| M| 80|
9) exit
Please enter your choice:
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Database Technical Soloution.exe
5| Poland national team home shirt 2023/24 | L| 45|
6| Argentina national team home shirt 2023/24 | M| 60|
7| Man city home shirt 2023/24| M| 80|
9) exit
Please enter your choice:
1
what quantity of this product do you want to buy
1
which customer is making the order

CustomerID| FirstName| LastName|
1| Bob| Shearer|
2| Alan| Shaw|
3| Guy| Walker|
4| Paul| Blart|
5| Luke| Bennet|
6| Brian| Taylor|
7| Joe| Ward|
8| Theo| Warne|
9| Declan| Simmonds |
10| Neil| Warlock|
11| John| McGinn|
12| Marshal| Kane|
13| Boris| Banks|
14| Fred| Keane|
15| Barry| Alan|
16| test| Salmon|
17| test| test|
Please enter your choice:
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical
9| Declan| Simmonds |
10| Neil| Warlock|
11| John| McGinn|
12| Marshal| Kane|
13| Boris| Banks|
14| Fred| Keane|
15| Barry| Alan|
16| test| Salmon|
17| test| test|
Please enter your choice:
17
would you like to add more to this order?
1)yes
2)no
1
which product would you like to order

ProductID| Name| size| Price|
1| southampton FC Home shirt 2023/24| M| 50|
2| southampton FC away shirt 2023/24| M| 36|
3| Arsenal FC home shirt 2023/24| L| 40|
4| Real Madrid FC home shirt 2023/24| S| 70|
5| Poland national team home shirt 2023/24 | L| 45|
6| Argentina national team home shirt 2023/24 | M| 60|
7| Man city home shirt 2023/24| M| 80|
9) exit
Please enter your choice:
2
what quantity of this product do you want to buy
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technica
15| Barry| Alan|
16| test| Salmon|
17| test| test|
Please enter your choice:
17
would you like to add more to this order?
1)yes
2)no
1
which product would you like to order

ProductID| Name| size| Price|
1| southampton FC Home shirt 2023/24| M| 50|
2| southampton FC away shirt 2023/24| M| 36|
3| Arsenal FC home shirt 2023/24| L| 40|
4| Real Madrid FC home shirt 2023/24| S| 70|
5| Poland national team home shirt 2023/24 | L| 45|
6| Argentina national team home shirt 2023/24 | M| 60|
7| Man city home shirt 2023/24| M| 80|
9) exit
Please enter your choice:
2
what quantity of this product do you want to buy
1
would you like to add more to this order?
1)yes
2)no
2
press enter to continue
```

Order tables UI (same as non order tables)

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Database Technical Soloution.exe
MAIN MENU
which action do you want to complete?
-----
1)Non-order tables
2)Make an order
3)order tables
4)View all orders
5)view graphs and statistics
9)exit
Please enter your choice: 3

WARNING: editing either of these tables may cause bugs in the database if not done properly

which table would you like to view :
1)database order table
2)database order details table
9) exit
```

View all orders UI

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Database Technical Soloution.exe
would you like to view all orders?
1)Yes
2)No
1
all orders:

OrderID| Full Name| Product Name| Price| city| OrderDate| ClubName| quantity| size|
1| Bob Shearer| southamton FC Home shirt 2023/24| 50| brighton| 24/05/2024| Southamton FC| 1| M|
2| Alan Shaw| southamton FC Home shirt 2023/24| 50| Newcastle| 02/06/2024| Southamton FC| 1| M|
3| Bob Shearer| southamton FC away shirt 2023/24| 36| brighton| 03/06/2024| Southamton FC| 1| M|
4| Guy Walker| Real Madrid FC home shirt 2023/24| 70| Weymouth| 03/06/2024| Real Madrid FC| 1| S|
5| Paul Blart| Arsenal FC home shirt 2023/24| 40| Portsmouth| 05/06/2024| Arsenal FC| 1| L|
6| Paul Blart| Real Madrid FC home shirt 2023/24| 70| Portsmouth| 25/06/2024| Real Madrid FC| 1| S|
7| Luke Bennet| Real Madrid FC home shirt 2023/24| 70| Blackburn| 26/06/2024| Real Madrid FC| 1| S|
8| Brian Taylor| southamton FC away shirt 2023/24| 36| London| 26/06/2024| Southamton FC| 1| M|
9| Joe Ward| southamton FC Home shirt 2023/24| 50| London| 27/06/2024| Southamton FC| 1| M|
10| Theo Warne| southamton FC Home shirt 2023/24| 50| Bournemouth| 01/07/2024| Southamton FC| 1| M|
11| Theo Warne| southamton FC Home shirt 2023/24| 50| Bournemouth| 01/07/2024| Southamton FC| 1| M|
12| Marshal Kane| Man city home shirt 2023/24| 80| London| 17/09/2024| Man city FC| 1| M|
13| Neil Warlock| Poland national team home shirt 2023/24 | 45| Southamton| 17/09/2024| Poland National team| 1| L|
14| Bob Shearer| southamton FC Home shirt 2023/24| 50| brighton| 01/10/2024| Southamton FC| 1| M|
15| Alan Shaw| southamton FC away shirt 2023/24| 36| Newcastle| 01/10/2024| Southamton FC| 2| M|
16| Boris Banks| Argentina national team home shirt 2023/24 | 60| Manchester| 14/10/2024| Argentina National Team| 2| M|
17| Barry Alan| Poland national team home shirt 2023/24 | 45| Manchester| 14/10/2024| Poland National team| 1| L|
17| Barry Alan| Real Madrid FC home shirt 2023/24| 70| Manchester| 14/10/2024| Real Madrid FC| 2| S|
18| Barry Alan| Arsenal FC home shirt 2023/24| 40| Manchester| 15/10/2024| Arsenal FC| 1| L|
19| Boris Banks| Arsenal FC home shirt 2023/24| 40| Manchester| 22/10/2024| Arsenal FC| 1| L|
20| Bob Shearer| Poland national team home shirt 2023/24 | 45| brighton| 22/10/2024| Poland National team| 2| L|
20| Bob Shearer| Real Madrid FC home shirt 2023/24| 70| brighton| 22/10/2024| Real Madrid FC| 1| S|
20| Bob Shearer| southamton FC away shirt 2023/24| 36| brighton| 22/10/2024| Southamton FC| 1| M|
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLU
would you like to view all orders?
1)Yes
2)No
2
would you like to view certain orders:
1)search orders from a certain customer
2)All orders that include a certain team
3)All orders with a certain price
4)All orders from a certain city
9) exit
Please enter your choice:
-
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Database Technical Soloution.exe
would you like to view all orders?
1)Yes
2)No
2
would you like to view certain orders:
1)search orders from a certain customer
2)All orders that include a certain team
3)All orders with a certain price
4)All orders from a certain city
9) exit
Please enter your choice:
1
search for a certain customers order by name(not case sensitive):
bob
Order:1      24/05/2024
-----
Bob Shearer    brighton

southampton FC Home shirt 2023/24(Southampton FC)    £50      size:M      quantity:1

Order:3      03/06/2024
-----
Bob Shearer    brighton

southampton FC away shirt 2023/24(Southampton FC)    £36      size:M      quantity:1
```

View graphs and statistics UI

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLU
MAIN MENU
which action do you want to complete?
-----
1)Non-order tables
2)Make an order
3)order tables
4)View all orders
5)view graphs and statistics
9)exit
Please enter your choice: 5

what action would you like to complete
1)view a graph
2)view statistics
9)exit to main menu
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Te
MAIN MENU
which action do you want to complete?
-----
1)Non-order tables
2)Make an order
3)order tables
4)View all orders
5)view graphs and statistics
9)exit
Please enter your choice: 5

what action would you like to complete
1)view a graph
2)view statistics
9)exit to main menu
1
which graph would you like to view
1) The sum of profit from each day
2) The price of the shirt that was sold each day
3) The total products each day
4) The total amount of customers each day
5) The total amount of orders each day
9) exit to main menu
```

```
G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Database Technical Soloution.exe
MAIN MENU
which action do you want to complete?
-----
1)Non-order tables
2)Make an order
3)order tables
4)View all orders
5)view graphs and statistics
9)exit
Please enter your choice: 5

what action would you like to complete
1)view a graph
2)view statistics
9)exit to main menu
2
what stat would you like to view
1)Gets the total amount of orders each customer has made
2)Gets the full names of the customers who have orders with prices larger than the average prices of all the shirts
3)Gets the product with the most sales
4)Gets the name and address of all customers on the database who have not made an order
9)exit
```

2.2.4 Pseudo Code for the general Error Handling System for User Inputs

The error/exception handling will need to be able to tell the user what they did wrong if the program does crash. However, we want to minimise the program crashing as much as possible and instead tell the user to try again and tell them what they did wrong. The code I have decided to use will make use of a while loop and a “try catch” system. A while loop will always be true until no error is detected. If an exception (the user enters something other than a number) occurs within the try brackets then the code will redirect to the catch statement, which will tell the user there was an exception and the probable reason for it. If there isn't an exception but the user enters a number that is bigger than the available menu options then they will loop through another while loop until a menu option is selected. This basic algorithm is used throughout the program whenever the user enters data. There are some changes made to suit each prompt but these are often minor.

```

bool error → true
while(error = true)
{
    Error = false
    Try
    {
        print("program prompts user for input (must be 1 or 2)")
        MenuChoice → int.parse(readline)
        Error → false
        while(MenuChoice != 1 AND MenuChoice !=2)
        {
            print("please enter a valid option (1,2)")
            MenuChoice → int.parse(readline)
            Error = false
        }
    }
    Catch
    {
        print("please enter a number (1,2)")
        Error → true
    }
}

```

2.3 Database Design

2.3.1 Queries

These are a few of the more complex queries that would be difficult for the user to enter but retrieve valuable information

Get total amount of orders each customer has made

```
SELECT (FirstName || ' ' || LastName) AS 'fullname', count(Orders.CustomerID) AS  
'total'  
FROM customer , Orders  
WHERE customer.CustomerID = Orders.CustomerID  
GROUP BY Orders.CustomerID
```

Get all the customers who made an order that had a price above average

```
SELECT (FirstName || ' ' || LastName) AS 'full name', price AS 'prices larger than  
average', (SELECT round(avg(price)) from Products) as 'average'  
FROM OrderDetails, Orders, customer , Products  
WHERE OrderDetails.OrderID = Orders.OrderID  
and Products.ProductID = OrderDetails.ProductID  
AND Orders.CustomerID = customer.CustomerID  
AND price > (SELECT avg(price) from Products)
```

Get the most popular product

```
SELECT Name, count(Name) AS 'number of sales'  
from Orders, Products , OrderDetails  
WHERE OrderDetails.ProductID = Products.ProductID  
AND OrderDetails.OrderID = Orders.OrderID  
GROUP BY Name ORDER BY count(Name) DESC
```

Get all customers who have never made an order

```
SELECT (FirstName || ' ' || LastName) AS 'fullname', Address  
FROM customer, Orders  
WHERE customer.CustomerID NOT IN (SELECT Orders.CustomerID FROM  
customer, Orders GROUP BY Orders.CustomerID)  
GROUP BY customer.CustomerID
```

Select all orders

```
SELECT Orders.OrderID, FirstName || ' ' || LastName as 'Full Name', Name as  
'Product Name', Products.price, city, OrderDate, ClubName, quantity, size
```

```
FROM customer , Products,Orders ,FootballClub,OrderDetails
WHERE Products.ProductID = OrderDetails.ProductID
AND Orders.OrderID = OrderDetails.OrderID
AND FootballClub.clubID = Products.clubID
AND customer.CustomerID = Orders.CustomerID
GROUP BY OrderDetails.OrderDetailsID
```

2.3.2 SQL Queries that use Data Inputted by the Program

These are queries that use a variable from the program incorporated into the SQL command. The variable is almost always something that has been entered by the user into the console. The SQLite library reads the parameters within the command the same way the Console.WriteLine function does in C# (It reads the whole parameter as a string including variables and their values). This makes entering variables of the correct data type into an SQL command possible.

Word in **blue** = variable from the C# program.

Get sum of profit from each day of that chosen month

```
SELECT sum(price),OrderDate
FROM customer,Orders,OrderDetails ,Products
WHERE customer.CustomerID = Orders.CustomerID
AND Orders.OrderID = OrderDetails.OrderID
AND OrderDetails.ProductID = Products.ProductID
AND substr(OrderDate,4,2)="" + monthString + ""
GROUP BY OrderDate
```

Search for orders made by a certain customer

```
SELECT Orders.OrderID, FirstName || ' ' || LastName as 'Full Name',Name as
'Product Name',Products.price,city,OrderDate,ClubName,quantity,size FROM
customer , Products,Orders ,FootballClub,OrderDetails WHERE
Products.ProductID = OrderDetails.ProductID AND Orders.OrderID =
OrderDetails.OrderID AND FootballClub.clubID = Products.clubID AND
customer.CustomerID = Orders.CustomerID AND FirstName || ' ' || LastName like""
+ search + "%" GROUP BY OrderDetails.OrderDetailsID
```

Search for orders from a certain club

```
SELECT Orders.OrderID, FirstName || ' ' || LastName as 'Full Name',Name as
'Product Name',Products.price,city,OrderDate,ClubName,quantity,size FROM
customer , Products,Orders ,FootballClub,OrderDetails WHERE
Products.ProductID = OrderDetails.ProductID AND Orders.OrderID =
OrderDetails.OrderID AND FootballClub.clubID = Products.clubID AND
```

```
customer.CustomerID = Orders.CustomerID AND ClubName like'" + search + "%'  
GROUP BY OrderDetails.OrderDetailsID
```

Search for orders at a certain price

```
SELECT Orders.OrderID, FirstName || ' ' || LastName as 'Full Name',Name as  
'Product Name',Products.price,city,OrderDate,ClubName,quantity,size FROM  
customer , Products,Orders ,FootballClub,OrderDetails WHERE  
Products.ProductID = OrderDetails.ProductID AND Orders.OrderID =  
OrderDetails.OrderID AND FootballClub.clubID = Products.clubID AND  
customer.CustomerID = Orders.CustomerID AND Price like'" + search + "%'  
GROUP BY OrderDetails.OrderDetailsID
```

Search for orders at a certain city

```
SELECT Orders.OrderID, FirstName || ' ' || LastName as 'Full Name',Name as  
'Product Name',Products.price,city,OrderDate,ClubName,quantity,size FROM  
customer , Products,Orders ,FootballClub,OrderDetails WHERE  
Products.ProductID = OrderDetails.ProductID AND Orders.OrderID =  
OrderDetails.OrderID AND FootballClub.clubID = Products.clubID AND  
customer.CustomerID = Orders.CustomerID AND city like'" + search + "%'  
GROUP BY OrderDetails.OrderDetailsID
```

Get the last row of data entered into the customer table

```
SELECT * FROM " tableID[ID] " WHERE CustomerID =(SELECT max(customerID)  
FROM customer)
```

Enter order data into the order table

```
INSERT INTO orders VALUES("'" + orderID + "'," + customerChoice + "'," +  
DateTime.Now.ToString().Substring(0, 10) + "'")
```

Get the sum of profit from each day (data to be used by the scatter plot)

```
SELECT sum(price),OrderDate FROM customer,Orders,OrderDetails ,Products  
WHERE customer.CustomerID = Orders.CustomerID AND Orders.OrderID =  
OrderDetails.OrderID AND OrderDetails.ProductID = Products.ProductID AND  
substr(OrderDate,4,2)='" + monthString + "' GROUP BY OrderDate
```

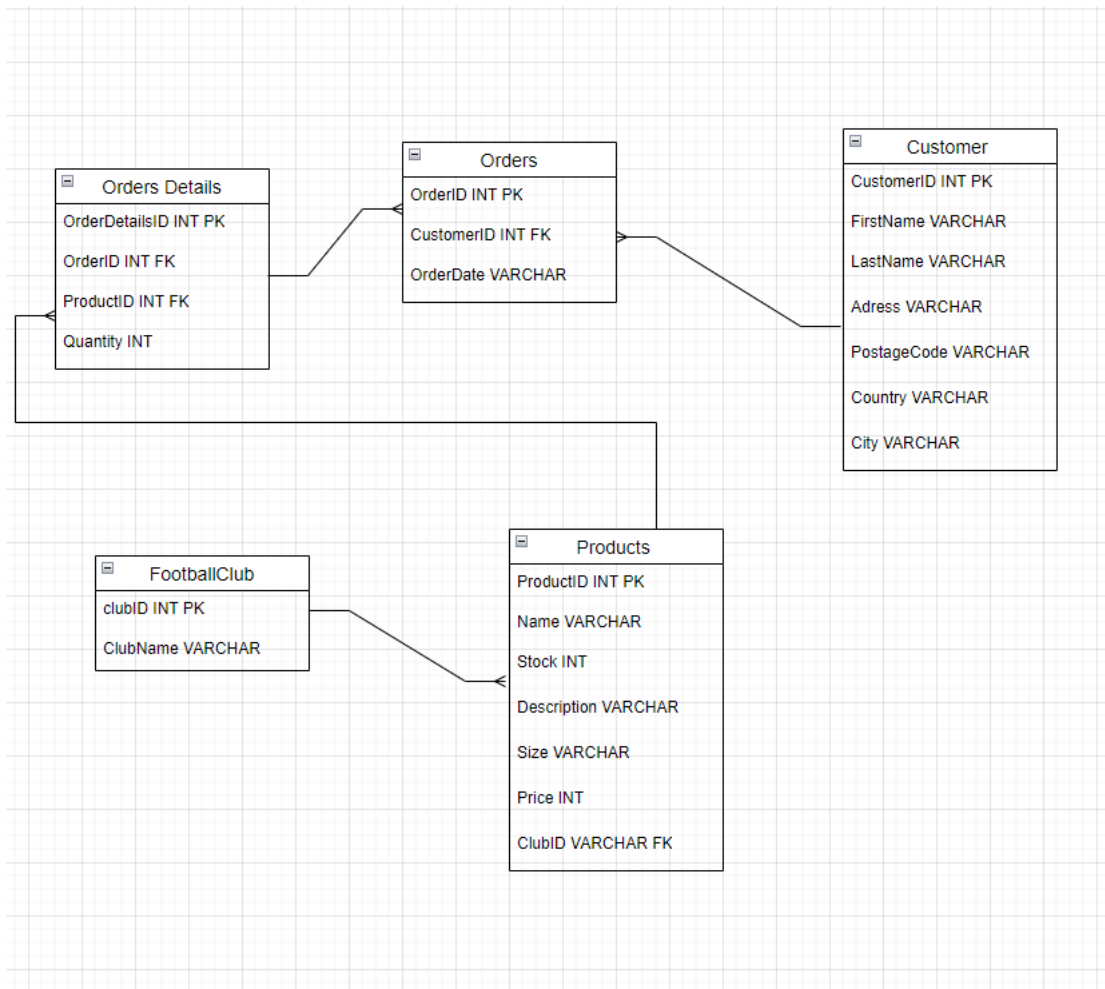
Get the price of the shirt sold from each day (data to be used by the scatter plot)

```
SELECT price, OrderDate from Orders ,OrderDetails, Products WHERE  
substr(OrderDate,4,2)='" + monthString + "' AND Products.ProductID =  
OrderDetails.ProductID AND Orders.OrderID = OrderDetails.OrderID
```

2.3.3 Table set up

Entity relationship diagram

This more detailed ER diagram shows how all the tables are connected to each other within the database and what data types they contain. You can clearly see that there are only one to many relationships and there are no non-key dependencies meaning that all the requirements for 3rd normal form have been fulfilled.



Relationships

customers(customerID, FirstName, LastName, Address, PostageCode, Country, City)

Products(ProductID, Name, stock, description, size, price, ClubID*)

Order(OrderID, CustomerID*, orderDate)

OrderDetails(OrderDetailsID, OrderID*, ProductID*, quantity)

FootballClub(clubID, ClubName)

2.3.4 Tables

The following screenshots are from DB browser, which is a database viewing application. This will help you get a more formatted view of the database.

The customer table

DB Browser for SQLite - G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Football shirt database.c

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project Attach Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: customer Filter in any column

	CustomerID	FirstName	LastName	Address	PostageCode	Country	city
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Bob	Shearer	14 Marlon Cresent	SO40RDX	United Kingdom	brighton
2	2	Alan	Shaw	17 Penny Lane	SO10PQX	United Kingdom	Newcastle
3	3	Guy	Walker	69 montgomery road	SO40TYB	United Kingdom	Weymouth
4	4	Paul	Blart	89 ivy lane	PO67MXY	United Kingdom	Portsmouth
5	5	Luke	Bennet	16 Dudley Hill	BL542BX	United Kingdom	Blackburn
6	6	Brian	Taylor	10 MouseHole Lane	LO412TU	United Kingdom	London
7	7	Joe	Ward	11 MouseHole Lane	LO422TU	United Kingdom	London
8	8	Theo	Warne	12 Alfred Road	BO452BG	United Kingdom	Bournemouth
9	9	Declan	Simmonds	12 Alan Road	SO128BE	United Kingdom	Southamton
10	10	Neil	Warlock	27 Happy Road	SO135RX	United Kingdom	Southamton
11	11	John	Mcginn	89 Victoria Lane	LO788RY	United Kingdom	London
12	12	Marshal	Kane	48 Stanley High Street	LO12BGT	United Kingdom	London
13	13	Boris	Banks	17 Church Road	Ma26FWA	United Kingdom	Manchester
14	14	Fred	Keane	89 Bridge Street	SO402BE	United Kingdom	Southampton
15	15	Barry	Alan	41 Victoria road	MA457BG	United Kingdom	Manchester
16	16	test	Salmon	11 Manchester Road	SO905HE	United Kingdom	Southamton
17	17	test	test	test	test	test	test

1 - 17 of 17 Go to: 1

The order table

DB Browser for SQLite - G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Football shirt databa

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project Attach Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: Orders Filter in any column

	OrderID	CustomerID	OrderDate
	Filter	Filter	Filter
1	1	1	24/05/2024
2	2	2	02/06/2024
3	3	1	03/06/2024
4	4	3	03/06/2024
5	5	4	05/06/2024
6	6	4	25/06/2024
7	7	5	26/06/2024
8	8	6	26/06/2024
9	9	7	27/06/2024
10	10	8	01/07/2024
11	11	8	01/07/2024
12	12	12	17/09/2024
13	13	10	17/09/2024
14	14	1	01/10/2024
15	15	2	01/10/2024
16	16	13	14/10/2024
17	17	15	14/10/2024
18	18	15	15/10/2024

1 - 19 of 29 Go to: 1

The order details table

DB Browser for SQLite - G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Football shirt data

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project Attach Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: OrderDetails

	OrderDetailsID	OrderID	ProductID	quantity
	Filter	Filter	Filter	Filter
1	1	1	1	1
2	2	2	1	1
3	3	3	2	1
4	4	4	4	1
5	5	5	3	1
6	6	6	4	1
7	7	7	4	1
8	8	8	2	1
9	9	9	1	1
10	10	10	1	1
11	11	11	1	1
12	12	12	7	1
13	13	13	5	1
14	14	14	1	1
15	15	15	2	2
16	16	16	6	2
17	17	17	5	1
18	18	17	4	2

1 - 19 of 36

Go to: 1

The products table

DB Browser for SQLite - G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Football shirt database.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project Attach Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: Products Filter in any column

	ProductID	Name	Stock	Description	size	Price	clubID
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	southampton FC Home shirt 2023/24	10	red and white stiped southampton home jersey	M	50	1
2	2	southampton FC away shirt 2023/24	10	white with blue trim southampton away jersey	M	36.99	1
3	3	Arsenal FC home shirt 2023/24	5	read and white with red trim Aresenal home jersey	L	40	2
4	4	Real Madrid FC home shirt 2023/24	5	white with gold trim real madrid home jersey	S	70	3
5	5	Poland national team home shirt 2023/24	5	White and red poland home jersey	L	45	4
6	6	Argentina national team home shirt 2023/24	10	Blue and white striped with black trim Argentina ...	M	60	5
7	7	Man city home shirt 2023/24	10	Light blue with white trim	M	80	8

1 - 7 of 7 Go to: 1

The football club table

DB Browser for SQLite - G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Football shirt database.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project Attach Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: FootballClub Filter in any column

	clubID	ClubName
	Filter	Filter
1	1	Southampton FC
2	2	Arsenal FC
3	3	Real Madrid FC
4	4	Poland National team
5	5	Argentina National Team
6	6	Turkey national team
7	7	West Ham FC
8	8	Man city FC

1 - 8 of 8 Go to: 1

The stats table

DB Browser for SQLite - G:\My Drive\COMPUTER SCIENCE NEA\TECHNICAL SOLUTION\Database Technical Soloution\bin\Debug\Football shirt database.

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project Attach Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: stats Filter in any column

	totalProduct	totalCustomer	totalOrder	DateOfEntry
	Filter	Filter	Filter	Filter
1	3	3	4	24/06/2024
2	6	9	11	30/08/2024
3	6	11	11	16/09/2024
4	7	11	11	17/09/2024
5	7	12	11	17/09/2024
6	7	14	13	17/09/2024
7	7	14	13	18/09/2024
8	7	15	13	20/09/2024
9	7	15	13	23/09/2024
10	7	16	13	30/09/2024
11	7	16	17	15/10/2024
12	7	16	21	22/10/2024
13	7	16	21	22/10/2024
14	7	16	21	22/10/2024
15	7	16	22	05/11/2024
16	7	16	22	05/11/2024
17	7	16	22	12/11/2024
18	7	16	22	12/11/2024

1 - 19 of 37 Go to: 1

2.4 Analytics and their Representation

2.4.1 Least Squares Regression

Least squares regression is a way of predicting future data entries by finding a regression line or a line of best fit, then you can enter a target Y value and find out by what X value will you reach your target Y value and the same is true the other way round (using a target X). You need to find the equation of the regression line ($Y=MX+C$) and then use the Y-intercept (C), the gradient and the target Y value (we will be predicting the X value for this program) in the equation $X \text{ prediction} = X \text{ Target} - C / M$. The equation for the regression line (future line of best fit labelled as m) and the y intercept (labelled as c).

$$m = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$c = \bar{y} - m\bar{x}$$

2.4.2 Pseudo Code of Least Square Regression

This pseudo code shows how the algorithm would be implemented into my program. There would first be a for loop that would iterate through the length of the data inputs(n) (which would be the same for X and Y) and then find the sum of the X and Y values multiplied together, the sum of all the X values, the sum of all the Y values and the sum of the X values squared. To find M, which is the gradient of the line, we multiply n (the length of the data inputs) by the sum of x and y multiplied together minus the sum of X multiplied by the sum of Y. We then divide the number by n multiplied by the sum of X squared and then takeaway the sum of x squared. To find C, which is the Y intercept, we do the sum of the Y values minus the gradient (M) multiplied by the sum of the X values. This number is then divided by n. We can then print the equation of the line ($y = mx + c$) to the console and show the user what the figures for this equation are. The user is then prompted to give their target Y. To find the prediction of the x value the target number for Y (what the user just entered) takeaway C (Y intercept) divided by the gradient (M) will be the prediction for when the x value will match the target Y value.

$lengthOfDataInput \rightarrow n$

for (i = 0 , to n, i++)

{

sumxy += x[i] * y[i]

sumx += x[i]

sumy += y[i]

sumXSquared += x[i] * x[i]

}

$m \rightarrow (n * sumxy - (sumx * sumy)) / (n * sumXSquared - (sumx * sumx))$

$c \rightarrow (sumy - m * sumx) / n$

print("y = mx + c:")

print("y = " + m + " x + " , c)

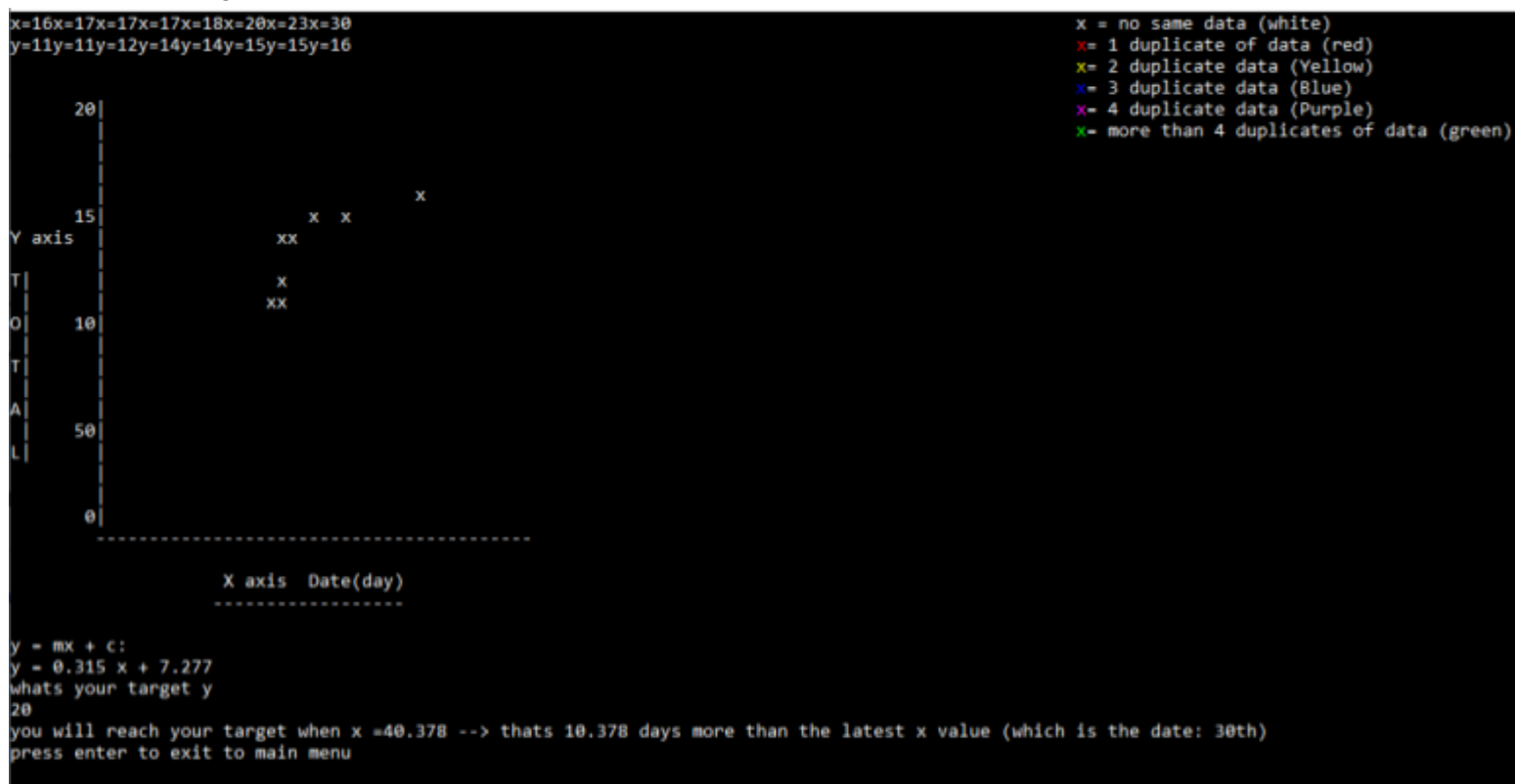
print("what is your target y")

yTarget $\rightarrow$ (ReadLine)

xPredict $\rightarrow$ (yTarget - c) / m

print("you will reach your target when x =" + xPredict)

This screenshot shows how the user will use the least square regression part of the program.

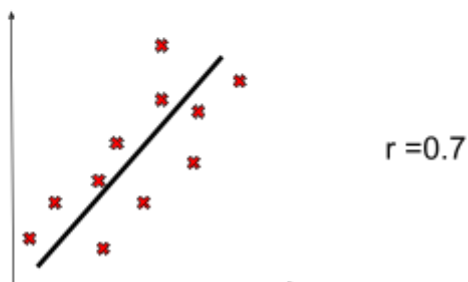
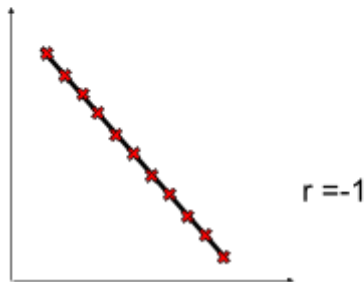
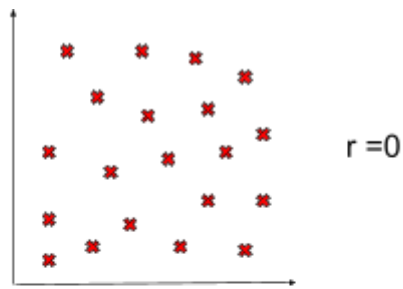
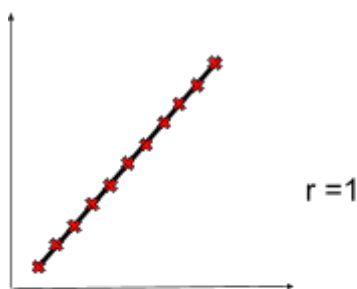


2.4.3 Product Moment Correlation Coefficient

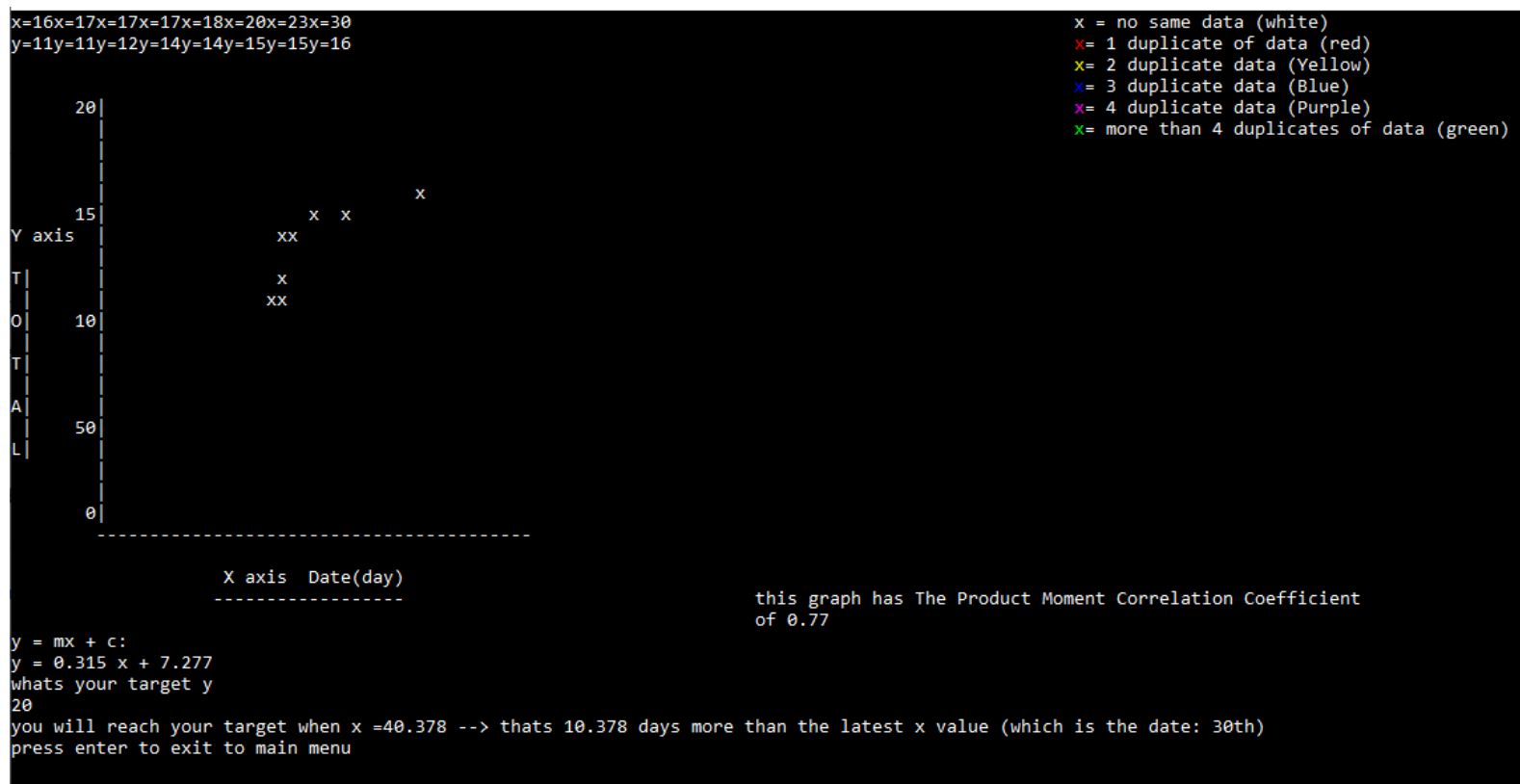
The product moment correlation coefficient is a measure of the linear correlation between two sets of data - in this case, X and Y. Unlike least squares regression the user does not need to enter any number and it just shows them how close the data is together and what way the data is trending. To integrate this into the program we use a code similar to the least squares regression code but this time we obviously use the Pmcc equation.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

This graph shows various datasets and their PMCC value(r) calculated, which can be any number between 1 and -1 based on the correlation of the data. If there is no correlation then r will be 0.



This screenshot shows how the user will use the least square regression and the Pmcc parts of the program together.



2.4.4 How duplicate data is dealt with

The reason we have an algorithm in the program to deal with duplicate data is because the dataset the graph is using may include many duplicates and this will not be represented by the graph and this may confuse the user. The objective of the duplicate finder algorithm is to change the colour of the x (the data point printed onto the graph) depending on the amount of times that data is duplicated. A regular white x means that the data (coordinate on the graph) is unique and is only seen in the dataset once. If it is red that means there is 1 other datapoint with those coordinates. If it is yellow there are 2 other duplicates. If it is blue there are 3 other duplicates. If it is pink then there are 4 other duplicates. If there are more than 4 duplicates then it will be green.

Basic pseudo code of the algorithm

The pseudo code below shows how we iterate through the length of the datasets X and Y (n), which should be the same, and then add the X and Y values to the duplicate list. We then iterate through the duplicate list and check to see if the X and Y values are equal to the values in the duplicate list. If a duplicate is found then a count is incremented. Depending on the value of this number a different colour of x is printed to the console at the coordinates of the x and y value.

```

for (i → 0 , to n, i++ )
{
    duplicateList.add (x[ i ] .toString + y[ i ] .toString)
    for (i → 0 , to duplicateList .Length, i++ )
    {
        If (x[i] .toString + y[ i ] .toString = dupeList[j ] AND i != j)
        {
            duplicateCount ++
        }
    }
    if(duplicateCount =0)
    {
        CursorPosition → (x[ i ] , y[ i ])
        print ("x")
    }
    if(duplicateCount =1)
    {
        ForegroundColour → red
        CursorPosition → (x[ i ] , y[ i ])
        print ("x")
    }
    if(duplicateCount = 2)
    {
        ForegroundColour → yellow
        CursorPosition → (x[ i ] , y[ i ])
        print ("x")
    }
    if(duplicateCount =3)
    {
        ForegroundColour → blue
        CursorPosition → (x[ i ] , y[ i ])
        print ("x")
    }
    if(duplicateCount =4)
    {
        ForegroundColour → purple
        CursorPosition → (x[ i ] , y[ i ])
        print ("x")
    }
    if(duplicateCount > 4)
    {
        ForegroundColour → green
        CursorPosition → (x[ i ] , y[ i ])
        print ("x")
    }
    ForegroundColour → white}

```

2.4.5 The graphs chosen and why

The purpose of the least Squares Regression graph is to help the end user to see possible future data entries. This helps the end user make informed business decisions that will hopefully help him grow his business. I also chose least squares regression because it is a relatively simple equation that can easily be implemented in C# (or any programming language).

I also chose to implement product moment correlation coefficient because it shows the end user how accurate the least squares regression prediction is likely to be. The closer the PMCC value is to 1 or -1 the more accurate the least squares regression prediction should be. This helps the user decipher whether it is worth listening to the analytics or if the data is too scattered from which to draw a useful conclusion.

3 Testing

A testing video is attached to my submission as a video file.

Test No.	Description	Test Data	Expected Outcome	Test Outcome	Pass /Fail	Evidence / Timestamp
A) Create an SQL database that stores and displays information for the business						
1	Test to see if the program can display the Customer table to the user. (General System test)	inputs:1,1,1	Program displays the table with all the contents of the customer table	Program displays the table with all the contents of the customer table	pass	00:07
2	Program closes when you press exit in main menu.	Selecting exit at the main menu (9)	Program closes	Program closes	pass	00:14
B)The program will validate the users input to avoid exceptions/errors						
3	See if the program crashes when entering a string into the main menu. (robustness testing)	Inputs: "abc"	error must be a number (1,2,3,4,5,6,7,8 or 9 to exit)	error must be a number (1,2,3,4,5,6,7,8 or 9 to exit)	pass	00:17
4	See if the program breaks when entering an integer larger than 9 (not on the main menu) into the main menu. (robustness testing)	Inputs: 53	please enter a valid choice (1,2,3,4,5,6,7,8 or 9 to exit)	please enter a valid choice (1,2,3,4,5,6,7,8 or 9 to exit)	pass	00:21
5	See if the program breaks entering a	Inputs:2~#abc	error must be a number (1,2,3,4,5,6,7	error must be a number (1,2,3,4,5,6,7	pass	00:26

	string of numbers symbols and letters. (robustness testing)		,8 or 9 to exit)	,8 or 9 to exit)		
6	Enter a string into the main menu and then another string and then a number larger than 9 (not on the main menu). (robustness testing)	"Abc" "Abc" 525	error must be a number (1,2,3,4,5,6,7 ,8 or 9 to exit) Twice and please enter a valid choice (1,2,3,4,5,6,7 ,8 or 9 to exit) Once	error must be a number (1,2,3,4,5,6,7 ,8 or 9 to exit) Twice and please enter a valid choice (1,2,3,4,5,6,7 ,8 or 9 to exit) Once	pass	00:29
C) Add functionality for the table						
7	Insert values into the database (customer table).	Inputs: 1,1,2,test,test, 57 test road, SO12JRT,UK, Southampton	press enter to return to main menu and values have been inserted	press enter to return to main menu and values have been inserted	pass	00:41
8	Check that the ID has been auto incremented correctly.	Go to DB Check database to see if customerID has been incremented properly from last insert test	The new value has been incremented	The new value has been incremented	pass	01:04
9	Display the last row of data that was entered into the table.	Inputs: 1,1,2	16 Alan Salmon 11 Manchester Road SO905HE United Kingdom Southampton	16 Alan Salmon 11 Manchester Road SO905HE United Kingdom Southampton	pass	00:45
10	Add pre calculated data into the stats table.	Inputs: 1,4,2	press enter to return to main menu and stats	press enter to return to main menu and stats	pass	01:16

			have been added to the table	have been added to the table		
11	Make sure the stats table enters the correct stats (total customers).	Number of customers (counted from the database)	Number of customers outputted by the stats table is the same as the counted number	The numbers are the same	pass	01:52
12	Make sure the stats table enters the correct stats (total orders).	Number of orders (counted from the database)	Number of orders outputted by the stats table is the same as the counted number	The orders are the same	pass	02:03
13	Make sure the stats table enters the correct stats (total products).	Number of products (counted from the database)	The number of products outputted by the stats table is the same	The products are the same	pass	02:10
14	Make sure the stats table enters the correct stats (today's date).	Check today's date from the OS	The dates match	The dates match	pass	01:27
15	The user is able to add columns to the table (add a column to the product table).	Inputs: 1,1,4,1,"test"	A column is added to the database called test	A column is added to the database called test	pass	02:21, 03:34
16	Test validation when renaming the column. (robustness testing)	1,1,4,12	please enter a valid number	please enter a valid number	pass	02:33
17	The user is able to rename columns from	Inputs: 1,1,4,3,8, "testRename"	The column is renamed to "testRename"	The column is renamed to "testRename"	pass	02:42

	the table (rename the newly added column).					
18	The user is able to remove columns from the table (remove that column from the table).	Inputs: 1,1,4,2,8	The column called test is removed from the database	The column called test is removed from the database	pass	02:54, 03:37
19	The user is able to edit data within the database (edit a name in the customer table).	Inputs: 1,1,3,2,16, "test"	The first name of ID 16 is changed to "test"	The first name of ID 16 is changed to "test"	pass	03:36
D) Create a way to search through orders to find specific records.						
20	Search for an order from a certain customer.	Inputs: 4,1,1"Barry"	Prints all orders from Barry to the console	Prints all orders from Barry to the console	pass	03:43
21	All orders can be printed to the console.	4,1	All orders and important data associated with them are shown to the user	All orders and important data associated with them are shown to the user	pass	03:45
22	Search for an order from a certain team.	Inputs: 4,1,2, "Arsenal FC"	Prints all orders that include a shirt from Arsenal.	Prints all orders that include a shirt from Arsenal.	pass	03:58
23	Search for an order of a certain price.	Inputs: 4,1,3, "70"	Prints all orders that are £70	Prints all orders that are £70	pass	04:13
24	Search for orders from a certain city.	Inputs: 4,1,4, "brighton"	Prints all orders that were made from brighton	Prints all orders that were made from Brighton	pass	04:25

25	Check wildcard works for searching for customers.	Inputs: 4,1,1”b”	Prints all orders for customers beginning with b	Prints all orders for customers beginning with b	pass	04:45
E) Make the order tables more useful to the user by adding:						
26	Make a single order using the make an order tab.	Inputs: 2,1,1,1,2	An order is added to the order table and Order details table accordingly	An order is added to the order table and Order details table accordingly	pass	05:03, 05:18
27	Make an order that includes more than 1 product.	Inputs: 2,1,1,1,1	An order is added to the order table and Order details table accordingly (with 2 products)	An order is added to the order table and Order details table accordingly (with 2 products)	pass	05:31, 05:47
28	Useful information (such as products and customers) about the orders are printed to the console while making an order.	Inputs: 2,1,1	When choosing a product all the products are shown and when choosing a customer all the customers are shown.	When choosing a product all the products are shown and when choosing a customer all the customers are shown.	pass	05:04
F) Create a graph in the console to represent useful data to customers and calculate useful numbers such as product moment correlation and least square regression.						
29	Show a test graph being printed to the console.	Inputs: 5,1,5,9	A graph is printed to the console	A graph is printed to the console	pass	06:00
30	Check that the gradient (m) is correct for the $y=mx+c$.	Inputs: 5,1,5,9	The result calculated is the same as the value printed to the console	The result calculated is the same as the value printed to the console	pass	06:12

	(complex algorithm testing)					
31	Check that the y intercept (c) is correct for the $y=mx+c$. (complex algorithm testing)	Inputs: 5,1,5,9	The result calculated is the same as the value printed to the console	The result calculated is the same as the value printed to the console	pass	06:12
32	Make sure the data from the month within the database is the same as the one in the program.	Inputs: 5,1,5,9	The listed values at the top are the same as the ones from the table	The listed values at the top are the same as the ones from the table	pass	06:37
33	Make sure the duplicate data is being coloured correctly. Test multiple inputs. (complex algorithm testing)	Inputs: 5,1,4,11 Dataset will be the month of November and the amount of customers on the database that is being used as test data	Test data showing duplicates of data is displayed correctly	Test data showing duplicates of data is displayed correctly	pass	06:58
34	Verify that the maths for the least squares regression is correct. (complex algorithm testing)	Inputs: 5,1,4,9, 30.857 data set: x= 16 17 17 17 18 20 23 30 Y = 11 11 12 14 14 15 15 16	The least square regression printed to the console is the same as the least square regression from the website calculator	The least squares regression printed to the console is the same as the least square regression from the website calculator	pass	07:13
35	Verify that the maths for the product moment	Inputs: 5,1,4,9 data set: x= 16 17 17	The calculated values(by the website) are	The calculated values (by the website)	pass	07:37

	correlation is correct. (complex algorithm testing)	17 18 20 23 30 Y = 11 11 12 14 14 15 15 16	the same as the values printed to the console.	are the same as the values printed to the console.		
36	Use a bigger data set (longer input list). (robustness testing)	Inputs: 5,1,4,9, Using test data 1 (from the program)	All the data is printed correctly (doesn't matter about how many data points just how big they are)	All the data is printed correctly	pass	07:59
37	Use a dataset that contains big numbers (3 digit). (robustness testing)	Inputs: 5,1,4,9, Using test data 2 (from the program)	The data that is within the bounds of the screen will be printed, the other values will be ignored	Program crashes	Fail	08:29
G) Use complex SQL to create useful statistics that connect multiple tables together. These statistics will include:						
38	Print the total amount of orders each customer has made.	Inputs:5,2,1	The printed value is the same as the calculated value.	The printed value is the same as the calculated value.	pass	08:48
39	Print the customers who have orders with prices larger than the average that is being printed.	Inputs:5,2,2	The printed value is the same as the calculated value.	The printed value is the same as the calculated value.	pass	08:56
40	Print the correct products with the most sales to the console.	Inputs:5,2,3	The printed value is the same as the calculated value.	The printed value is the same as the calculated value.	pass	09:00
41	Print the name and address of	Inputs:5,2,4	The printed value is the	The printed value is the	pass	09:07

	all customers on the database who have not made an order to the console.		same as the calculated value.	same as the calculated value.		
--	--	--	-------------------------------	-------------------------------	--	--

Link to video and QR code:

 DB NEA Testing video.mp4



Links used in the video:

<https://www.standarddeviationcalculator.io/correlation-coefficient-calculator>

<https://www.statskingdom.com/linear-regression-calculator.html>

4 Evaluation

4.1 Overall Effectiveness of the System

I believe my system has done a superb job at solving the objectives given to me by the end user, although I do believe that there are some crucial changes or improvements that need to be made. I have created an SQL database that stores important information about the business and the orders it receives. I have added most if not all of the important functionality of SQL into C#. I have made 2 order tables that are fully normalised and track important information regarding the orders. I have created a graph that plots data and shows their correlations and I have also handled any duplicate data by representing it as a different colour. I have also used complex SQL to create useful statistics and analytics regarding the business. However, I believe I could have improved the UI/ menu system and maybe used a better system in the console rather than entering in numbers and maybe even using a GUI. There were also certain moments where you could not go back to the menu if you were entering data. Also, there are other parts of the program where the user may be using a certain function multiple times and it takes you back to the main menu every time, which wastes a lot of time. Furthermore, I would like to have improved the graph instead of printing it out to the console I could have used a library or other program that links to C# to create better graphs (in terms of looks and accuracy). I also wish that I had just made a subroutine for error handling rather than copying and pasting the same code and modifying it for each input. I also added an auto-increment function that would automatically increment the IDs within the database, however, I later found out there is already a built-in function in SQL for auto-incrementing. However, this can be seen as a positive because it gives me more control over what I can input into the database and I could maybe modify it later. One minor issue with the program is the fact that when it prints data to the console the data isn't nicely lined up, which makes it difficult for the user to read the data. Finally, there is also a problem with the graph where if you use a dataset that contains big numbers bigger than the axis or the screen size then the program breaks. This is a major problem due to the fact that the axis isn't even very big and there is a high likelihood in the future when the end user is using my program that bigger numbers are introduced into the database. Also if the the difference between the biggest number and the smallest number is so big that it cannot be scaled to fit onto the screen properly then the program will break in this case as well.

4.2 Evaluation of Objectives

- 1) Create an SQL database that stores information for the business✓
I have created a SQL database that is connected to C# and contains information regarding the business.
- 2) Connect the SQL database to a programming language so that queries can easily be used in a program.
I have completed this objective
- 3) The database will store and display information to the user and contain 6 normalised tables:✓
 - a) Customer ✓
 - b) Order ✓
 - c) Order information (**now called Order Details in my program and database**)✓
 - d) Product ✓
 - e) Football club information (**now just called Football Club in my program and database**)✓
 - f) Stats✓**I have managed to create each of these tables and normalise them accordingly. One minor issue is that the column stock doesn't have any functionality. The user can still edit it like all other records but I maybe could have done something to make it easier to update.**
- 4) The program will check if user inputs are valid to avoid exceptions/errors✓
I have met this requirement, however, I could have done it better. Rather than copy and pasting the same error handling code I wish I had just made one subroutine and used that over and over again to save time and improve readability.
- 5) Functionality for the table such as:
 - a) Insert values into the database ✓
 - i) Make it so that before you insert values into the table it shows you the last row of data entered.✓**I have completed both of these objectives to a fine standard, however, when you enter data into the database you cannot exit to the main menu or go back so I could have added that in to improve upon my program.**
 - b) Calculate statistics for the database and put them in their own table for the user to see✓
I have completed this objective quite well because it does not allow the user to change the table and the only data entered into the table is stats that my program has calculated meaning in theory all the data is correct.
 - c) They will be able to alter the columns from all the tables in the database (apart from the stats table):✓
 - i) Add new columns ✓
 - ii) Remove columns ✓
 - iii) Rename columns ✓

I have completed this objective perfectly for the most part, however, when renaming the column you cannot exit or go back to the main menu, which forces the user to exit the program if they change their mind about changing the column (same problem as the insert objective).

- d) Add a way to edit data within the database from the program✓

I have completed this objective but I could improve on what I have done maybe by making the change easier to do and making it graphical (maybe making it so that you could click on the data/cell you want to edit and then edit it).

- e) Create a way to search through all orders to find specific records. Ways to search will include:✓
- i)search for orders from a certain customer✓
 - ii)search for orders that include a certain team✓
 - iii)search for orders with a certain price✓
 - iv)search for orders from a certain city✓

I have completed this objective well and I have used wildcards to make the search easy to use.

- 6) Create two order tables (orders and order information) :✓
- a) An order table, which is the top-level order information such as the customer ordering and the date✓
 - b) And Order details table being more details about the order itself and what it contains(product and quantity)✓
 - c) The tables are fully normalised ✓
 - d) Have a way to represent the two order tables in a way which is useful to the customer such as an "all orders table"✓
 - e) Make a way to add multiple products to one order✓

I have completed these objectives well.

- 7) Create a graph in the console to represent useful data to customers and calculate useful figures such as the product moment correlation and least square regression(the different graphs will represent each month).

These graphs will be :

- a) The sum of profit from each day✓
- b) The price of each shirt sold each day✓
- c) The total products each day✓
- d) The total amount of customers each day✓
- e) The total amount of orders each day✓

I have completed this objective well, however some datasets using bigger numbers can break the graph (it goes off the screen) and I should have made a way to prevent this. For the Y-axis, I divided the raw data by 10, however, what if the data is so big to the point where dividing by 10 it is still printed off of the screen? Furthermore, there is no limit on the size of the data lists entered.

- 8) Make sure any duplicate data is plotted in a different colour to non-duplicate data so that every datapoint is represented on the graph.✓

I completed this objective well.

9) Use complex SQL to create useful statistics that connect multiple tables together.

These statistics will include:

- a) The total amount of orders each customer has made✓
- b) The full names of the customers who have orders with prices larger than the average prices of all the shirts✓
- c) The product with the most sales✓
- d) The name and address of all customers on the database who have not made an order✓

I completed this objective well.

4.3 End User Feedback

Interviewer

Interviewee

Do you feel like you have full freedom over your database and can change it however you want?

Yes I feel like I can change everything I need to including the columns and the data stored underneath those columns as well. However I cannot add new tables which I feel may not be an issue now but in future if I want to expand and add other tables I will not be able to do so with your program.

Do you feel like the database is easier to use within my program or would you rather just edit it using a database browser?

I think your program makes it easier to use for the most part however I would much rather use a graphical user interface than a text based program. This would not only look better but also be easier to use especially for things like editing values in the database.

Do you feel like I have added all the functionality you want?

I think you have added pretty much everything I wanted but I think you could implement them better, for example a graphical interface would be great and also the graph could be represented better with both axis being labelled.

Do you feel like my program provides you with useful analytics and statistics?

I think the least squares regression and the other stats you provided such as customers who never made an order are successful in providing useful insights into the business and also give me a good idea of the future of my business. Overall I think this is one of the best parts of the program. However, I also think that the graphs should have been represented in a graphical system rather than a text-based one. I think this would improve the clarity of the graph and show more detail rather than just the general trend.

Do you think my graph has been successful at representing the data in a useful way?

Again, I think that creating a graph within a dedicated graphing software would be better than trying to print it to the console. I think this would make it nicer to look at / easier to understand which would improve clarity. I think the graph is useful for showing general trends but not so good for more detailed analysis.

Do you think that calculating least squares regression helps give useful business insights and if so is it also useful to have the product moment correlation coefficient alongside it?

Yes I believe this was a very good idea to have these 2 figures shown to the user together to show the prediction and the validity of the prediction. Sometimes when using other software when they give you a similar prediction they do not tell you how realistic the prediction is.

From this final interview, I can see that Finlay is overall happy with my program but feels that some changes could have been made to make it even better. Firstly it seems like implementing a graphical interface (using something like Windows Forms) would have been a much better choice than producing the entire program in the console. Furthermore, I could have represented the graphs in a graphical way if I had used Windows Forms. He also spoke about how there is no way to add a new table to the database and in hindsight, I definitely should have added this, however I did not deem it necessary when designing the program.

4.4 System Improvements

The biggest issue that I could solve within my system relatively easily would be to create one subroutine for error handling rather than repeating it over and over again and changing the prompt for the user each time. This would increase the readability of the code and make it easier to debug if any issues were to arise. Another minor fix that I discovered is that the field in products called stock does not have any functionality. I could have added functionality such as minus incrementing stock if an order includes that product or being able to add new stock. However, another major issue that the end user and I agree on is that the program would have been better off as a GUI (making use of Windows Forms). This would greatly improve the looks and therefore clarity of the program making the graphs more accurate and easy to read and making it easier to click and edit records in the database. Finally, one small issue with the program is that when entering data there is no way of going back to the page you were just on. This is due to the nature of text-based programs, however with a graphical program this problem could have been fixed. Also when you complete an action such as viewing a graph, you are forced back to the main menu when actually it is very likely you would want to view another graph so a slight update to the menu system may have been a good idea. Finally, the issue of the program breaking when using a dataset that contains large numbers can be fixed by using an algorithm that scales the whole dataset down to fit on the graph axis. Or maybe a small if statement that finds the largest and smallest data point from each of the data sets (x and y) and if there is too big of a gap then the program refuses to print any data to the console and advises the user to choose a different data set. This is already in place to a minor scale because by default numbers on the Y-axis are divided by 10 to fit on the screen - there is no need to do this for the X-axis due to the fact that it is always just the date which can only be 1 to 32 max. An expansion of this system may be a good fix for this issue. I would also have to limit the difference between the biggest and smallest numbers in the dataset so that the scaling system works.

5 Technical Solution

Directory of Technical Skills

The complex code may not be directly where the page number is, it may be in a subroutine below and used within the subroutine that the page number did lead to.

Libraries → **page 52**

Main → **page 52**

User picks which statistics they want to see (complex SQL in subroutines below) → **page 55**

User can make an order → **page 58**

User can add more to the same order → **page 63**

The program selects all orders for the user to see → **page 66**

The user can search through all of the orders by customer (they can also search by club, price and city as seen in the subroutines below) → **page 67**

The user can view a specific table, which was chosen earlier in the program → **page 73**

The user can edit a specific chosen table → **page 73**

The user can alter (add a new column, delete a column or rename a column) a table → **page 75**

Gets the last row of data entered into that chosen table → **page 80**

Every time the user adds data to a table that has a primary key, the primary key is automatically incremented → **page 85**

The user chooses which graph they want to see → **page 87**

The user chooses what data they want to see plotted on the scatter graph and the data is collected → **page 90**

The scatter graph is printed to the console → **page 94**

The product moment correlation coefficient is calculated → **page 100**

Least squares regression is calculated → **page 101**

The connection to the SQL database from C# → **page 105**

Code:

```
using System;
using System.Collections.Generic;
using System.Data.SQLite;
using System.Linq;

namespace Database_Technical_Soloution
{
    internal class Program
    {
        const int GraphWidth = 41;
        const int graphHight = 20;
```

```

const int graphDistFromTop = 4;
const int graphDistFromSide = 8;
const int graphmidpointY = 20 + graphDistFromTop;
static string[] tableID = new string[6];

static void Main(string[] args)
{
    Console.WindowWidth = Console.LargestWindowWidth - 80;
    tableID[0] = "customer";
    tableID[1] = "Orders";
    tableID[2] = "OrderDetails";
    tableID[3] = "Products";
    tableID[4] = "footballClub";
    tableID[5] = "stats";
    int choice = 0;
    int orderTableChoice = 0;

    while (choice != 9)
    {
        choice = GetMainMenuChoices();
        if (choice == 1)
        {
            switch (NonOrderTables())
            {
                case 1:
                    CustomerTable(0);
                    break;
                case 2:
                    ProductTable(3);
                    break;
                case 3:
                    FootballClubTable(4);
                    break;
                case 4:
                    StatsTable(5);
                    break;
                case 9:
                    break;
                default:
                    Console.WriteLine("error no table chosen");
                    break;
            }
        }
        else if (choice == 2)
        {
            MakeAnOrder();
        }
        else if (choice == 3)
        {

```

```

        Console.WriteLine("WARNING: editing either of these
tables may cause bugs in the database if not done properly");
        Console.WriteLine();
        bool error = true;
        while (error == true)
        {
            error = false;
            try
            {
                Console.WriteLine("which table would you like to
view :");

                Console.WriteLine("1)database order table");
                Console.WriteLine("2)database order details
table");

                Console.WriteLine("9) exit");
                orderTableChoice =
int.Parse(Console.ReadLine());
                error = false;
                while (orderTableChoice != 1 && orderTableChoice
!= 2 && orderTableChoice != 9)
                {
                    Console.WriteLine("please enter a valid
choice (1,2,9) ");
                    orderTableChoice =
int.Parse(Console.ReadLine());
                    error = false;
                }
            }
            catch
            {
                Console.WriteLine("error must be a number
(1,2,9)");

                Console.WriteLine();
                error = true;
            }
        }

        if (orderTableChoice == 1)
        {
            OrderTable(1);
        }
        else if (orderTableChoice == 2)
        {
            OrderDetailsTable(2);
        }
    }
    else if (choice == 4)
    {
        ViewAllOrders();
    }
    else if (choice == 5)

```

```

    {
        // graph and staticstics
        int ID = 0;
        bool error = true;
        while (error == true)
        {
            error = false;
            try
            {
                Console.WriteLine("what action would you like to
complete ");

                Console.WriteLine("1)view a graph ");
                Console.WriteLine("2)view statistics");
                Console.WriteLine("9)exit to main menu");
                ID = int.Parse(Console.ReadLine());
                error = false;
                while (ID != 1 && ID != 2 && ID != 9)
                {
                    Console.WriteLine("please enter a valid
choice (1,2,9) ");

                    ID = int.Parse(Console.ReadLine());
                    error = false;
                }
            }
            catch
            {
                Console.WriteLine("error must be a number
(1,2,9)");

                Console.WriteLine();
                error = true;
            }
        }
        //graph choice
        if (ID == 1)
        {
            GraphChoice(ID);
        }
        // statistic choice
        else if (ID == 2)
        {
            ShowStatistics(ID);
        }
    }
}

static int NonOrderTables()
{
    int ID = 0;
    Console.WriteLine("which table would you like to access?");
    Console.WriteLine("1)Customer table");

```

```

        Console.WriteLine("2)Product table");
        Console.WriteLine("3)Football club table");
        Console.WriteLine("4)Stats table");
        Console.WriteLine("9)Exit");
        return ID = int.Parse(Console.ReadLine());
    }
    static void ShowStatistics(int ID)
    {
        SQLiteConnection conn = CreateConnection();
        ID = 0;
        bool error = true;
        while (error == true)
        {
            error = false;
            try
            {
                Console.WriteLine("what stat would you like to view");
                Console.WriteLine("1)Gets the total amount of orders
each customer has made");
                Console.WriteLine("2)Gets the full names of the
customers who have orders with prices larger than the average prices of all
the shirts");
                Console.WriteLine("3)Gets the product with the most
sales");
                Console.WriteLine("4)Gets the name and address of all
customers on the database who have not made an order");
                Console.WriteLine("9)exit");
                ID = int.Parse(Console.ReadLine());
                error = false;
                while (ID != 1 && ID != 2 && ID != 3 && ID != 4 && ID !=
9)
                {
                    Console.WriteLine("please enter a valid choice
(1,2,3,4 or 9 to exit)");
                    ID = int.Parse(Console.ReadLine());
                    error = false;
                }
            }
            catch
            {
                Console.WriteLine("error must be a number (1,2,3,4 or 9
to exit)");
                Console.WriteLine();
                error = true;
            }
        }
        if (ID != 9)
        {
            if (ID == 1)
            {
                //Gets the total amount of orders each customer has made

```

```

        GetTotalAmountOfOrdersEachCustomerHasMade();
    }
    else if (ID == 2)
    {
        //Gets the full names of the customers who have orders
        with prices larger than the average prices of all the shirts
        GetAllCustomersWhoMadeAnOrderAboveAverage();
    }
    else if (ID == 3)
    {
        //most popular product
        GetMostPopularProduct();
    }
    else if (ID == 4)
    {
        //customers who never made an order
        GetCustomersWhoNeverMadeAnOrder();
    }
}

static void GetTotalAmountOfOrdersEachCustomerHasMade()
{
    SQLiteConnection conn = CreateConnection();
    SQLiteCommand cmd = new SQLiteCommand("SELECT (FirstName || '
'||LastName) AS 'fullname',count(Orders.CustomerID) AS 'total' FROM customer
, Orders WHERE customer.CustomerID = Orders.CustomerID GROUP BY
Orders.CustomerID", conn);
    SQLiteDataReader result = cmd.ExecuteReader();
    for (int i = 0; i < result.FieldCount; i++)
    {
        Console.Write(result.GetName(i) + "| ");
    }
    while (result.Read())
    {
        Console.WriteLine();

        for (int i = 0; i < result.FieldCount; i++)
        {
            Console.Write(result[i] + "| ");
        }
    }
    conn.Close();
    Console.WriteLine();
    Console.WriteLine();
    Console.WriteLine("press enter to return to main menu");
    Console.ReadKey();
}

static void GetAllCustomersWhoMadeAnOrderAboveAverage()
{
    SQLiteConnection conn = CreateConnection();

```

```

        SQLiteCommand cmd = new SQLiteCommand("SELECT (FirstName || ' ' ||
LastName) AS 'full name', price AS 'prices larger than average', (SELECT
round(avg(price)) from Products)as 'average' from
OrderDetails,Orders,customer ,Products WHERE OrderDetails.OrderID =
Orders.OrderID and Products.ProductID = OrderDetails.ProductID AND
Orders.CustomerID = customer.CustomerID AND price > (SELECT avg(price) from
Products)", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        for (int i = 0; i < result.FieldCount; i++)
        {
            Console.Write(result.GetName(i) + "| ");
        }
        while (result.Read())
        {
            Console.WriteLine();

            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i] + "| ");
            }
        }
        conn.Close();
        Console.WriteLine();
        Console.WriteLine();
        Console.WriteLine("press enter to return to main menu");
        Console.ReadKey();
    }
    static void GetMostPopularProduct()
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd = new SQLiteCommand("SELECT Name, count(Name)
AS 'number of sales' from Orders, Products ,OrderDetails WHERE
OrderDetails.ProductID = Products.ProductID AND OrderDetails.OrderID =
Orders.OrderID GROUP BY Name ORDER BY count(Name) DESC", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        for (int i = 0; i < result.FieldCount; i++)
        {
            Console.Write(result.GetName(i) + "| ");
        }
        while (result.Read())
        {
            Console.WriteLine();

            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i] + "| ");
            }
        }
        conn.Close();
        Console.WriteLine();
        Console.WriteLine();
    }

```

```

        Console.WriteLine("press enter to return to main menu");
        Console.ReadKey();
    }
    static void GetCustomersWhoNeverMadeAnOrder()
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd = new SQLiteCommand("SELECT (FirstName || '
        '||LastName) AS 'fullname', Address FROM customer,Orders WHERE
        customer.CustomerID NOT IN (SELECT Orders.CustomerID FROM customer,Orders
        GROUP BY Orders.CustomerID) GROUP BY customer.CustomerID", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        for (int i = 0; i < result.FieldCount; i++)
        {
            Console.Write(result.GetName(i) + "| ");
        }
        while (result.Read())
        {
            Console.WriteLine();

            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i] + "| ");
            }
        }
        conn.Close();
        Console.WriteLine();
        Console.WriteLine();
        Console.WriteLine("press enter to return to main menu");
        Console.ReadKey();
    }
    static void MakeAnOrder()
    {
        Console.Clear();
        SQLiteConnection conn = CreateConnection();
        int orderID = GetMaxIncrementNum(1);
        int count = GetFieldCountForProducts();
        int productChoice = 0;
        int customerChoice = 0;
        int quantityChoice = 0;
        int addExtraToOrder = 0;

        productChoice = GetProductChoice(count);
        if (productChoice != 9)
        {
            quantityChoice = GetQunatityChoice();
            customerChoice = GetCustomerChoice(count);

            SQLiteCommand cmd1 = new SQLiteCommand("INSERT INTO orders
            VALUES('" + orderID + "','" + customerChoice + "','" +
            DateTime.Now.ToString().Substring(0, 10) + "',')", conn);

```

```

        cmd1.ExecuteNonQuery();
        SQLiteCommand cmd2 = new SQLiteCommand("INSERT INTO
orderDetails VALUES('" + GetMaxIncrementNum(2) + "',' + orderID + "',' +
productChoice + "',' + quantityChoice + "',')", conn);
        cmd2.ExecuteNonQuery();

        addExtraToOrder = GetAddMoreToOrderChoice();
        while (addExtraToOrder == 1)
        {
            AddMoreToOrder(count, orderID);
            addExtraToOrder = GetAddMoreToOrderChoice();
        }
        Console.WriteLine("press enter to continue");
        Console.ReadKey();
    }
}

static int GetProductChoice(int count)
{
    SQLiteConnection conn = CreateConnection();
    int productChoice = 0;
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("which product would you like to order
");

            Console.WriteLine();

            SQLiteCommand cmd = new SQLiteCommand("SELECT
ProductID,Name,size,Price FROM Products", conn);
            SQLiteDataReader result = cmd.ExecuteReader();
            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result.GetName(i) + "| ");
            }
            while (result.Read())
            {
                Console.WriteLine();

                for (int i = 0; i < result.FieldCount; i++)
                {
                    Console.Write(result[i] + "| ");
                }
            }
            result.Close();
            Console.WriteLine();
            Console.WriteLine("9) exit");

```

```

        Console.WriteLine("Please enter your choice: ");
        productChoice = int.Parse(Console.ReadLine());
        error = true;

        for (int i = 1; i <= count; i++)
        {
            if (productChoice == i || productChoice == 9 ||
error == false)
            {
                error = false;
            }
        }
    }
    catch
    {
        Console.WriteLine("error must be a number (1,2,3,4 or 9
to exit)");
        Console.WriteLine();
        error = true;
    }
}
return productChoice;
}
static int GetFieldCountForProducts()
{
    SQLiteConnection conn = CreateConnection();
    int count = 0;
    SQLiteCommand cmd = new SQLiteCommand("SELECT
ProductID,Name,size,Price FROM Products", conn);
    SQLiteDataReader result = cmd.ExecuteReader();
    while (result.Read())
    {
        count++;
    }
    return count;
}
static int GetQunatityChoice()
{
    bool error = true;
    int quantityChoice = 0;
    error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("what quantity of this product do you
want to buy");
            quantityChoice = int.Parse(Console.ReadLine());
        }
        catch

```

```

        {
            Console.WriteLine("error must be a number (1,2,3,4 or 9
to exit)");
            Console.WriteLine();
            error = true;
        }
    }
    return quantityChoice;
}
static int GetCustomerChoice(int count)
{
    int customerChoice = 0;
    SQLiteConnection conn = CreateConnection();
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("which customer is making the order");
            Console.WriteLine();

            SQLiteCommand cmd = new SQLiteCommand("SELECT
CustomerID, FirstName, LastName FROM customer", conn);
            SQLiteDataReader result = cmd.ExecuteReader();
            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result.GetName(i) + "| ");
            }
            while (result.Read())
            {
                Console.WriteLine();

                for (int i = 0; i < result.FieldCount; i++)
                {
                    Console.Write(result[i] + "| ");
                }
                count++;
            }
            result.Close();
            Console.WriteLine();
            Console.WriteLine("Please enter your choice: ");
            customerChoice = int.Parse(Console.ReadLine());
            error = true;

            for (int i = 1; i <= count; i++)
            {
                if (customerChoice == i || customerChoice == 9)
                {
                    error = false;
                }
            }
        }
        catch { }
    }
}

```

```

        }
    }
    catch
    {
        Console.WriteLine("error must be a number (1,2,3,4 or 9
to exit)");
        Console.WriteLine();
        error = true;
    }
}
return customerChoice;
}
static int GetAddMoreToOrderChoice()
{
    int addExtraToOrder = 0;
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("would you like to add more to this
order?");
            Console.WriteLine("1)yes");
            Console.WriteLine("2)no");
            addExtraToOrder = int.Parse(Console.ReadLine());
            error = false;
            while (addExtraToOrder != 1 && addExtraToOrder != 2)
            {
                Console.WriteLine("please enter a valid choice
(1,2,3,4 or 9 to exit) ");
                addExtraToOrder = int.Parse(Console.ReadLine());
                error = false;
            }
        }
        catch
        {
            Console.WriteLine("error must be a number (1,2,3,4 or 9
to exit)");
            Console.WriteLine();
            error = true;
        }
    }
    return addExtraToOrder;
}
static void AddMoreToOrder(int count, int orderID)
{
    SQLiteConnection conn = CreateConnection();
    int productChoice = 0;
    int quantityChoice = 0;
    bool error = true;

```

```

while (error == true)
{
    error = false;
    try
    {
        Console.WriteLine("which product would you like to order
");
        Console.WriteLine();

        SQLiteCommand cmd = new SQLiteCommand("SELECT
ProductID,Name,size,Price FROM Products", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        for (int i = 0; i < result.FieldCount; i++)
        {
            Console.Write(result.GetName(i) + "| ");
        }
        while (result.Read())
        {
            Console.WriteLine();

            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i] + "| ");
            }
            count++;
        }
        result.Close();
        Console.WriteLine();
        Console.WriteLine("9) exit");
        Console.WriteLine("Please enter your choice: ");
        productChoice = int.Parse(Console.ReadLine());
        error = true;

        for (int i = 1; i <= count; i++)
        {
            if (productChoice == i || productChoice == 9)
            {
                error = false;
            }
        }
    }
    catch
    {
        Console.WriteLine("error must be a number (1,2,3,4 or 9
to exit)");
        Console.WriteLine();
        error = true;
    }
}
error = true;

```

```

while (error == true)
{
    error = false;
    try
    {
        Console.WriteLine("what quantity of this product do you
want to buy");
        quantityChoice = int.Parse(Console.ReadLine());
    }
    catch
    {
        Console.WriteLine("error must be a number (1,2,3,4 or 9
to exit)");
        Console.WriteLine();
        error = true;
    }
}
//only order details needed
SQLiteCommand cmd3 = new SQLiteCommand("INSERT INTO orderDetails
VALUES('" + GetMaxIncrementNum(2) + "',''" + orderID + "',''" + productChoice
+ "',''" + quantityChoice + "')", conn);
cmd3.ExecuteNonQuery();
}
static void ViewAllOrders()
{
    Console.Clear();
    SQLiteConnection conn = CreateConnection();
    int certainOrdersChoice = 0;
    int viewAllOrders = 0;
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("would you like to view all orders?

");
            Console.WriteLine("1)Yes");
            Console.WriteLine("2)No");
            viewAllOrders = int.Parse(Console.ReadLine());
            error = false;
            while (viewAllOrders != 1 && viewAllOrders != 2)
            {
                Console.WriteLine("please enter a valid choice (1,2)

");
                viewAllOrders = int.Parse(Console.ReadLine());
                error = false;
            }
        }
        catch

```

```

        {
            Console.WriteLine("error must be a number (1,2)");
            Console.WriteLine();
            error = true;
        }
    }
    if (viewAllOrders == 1)
    {
        SelectAllOrders();
    }
    error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("would you like to view certain
orders:");
            Console.WriteLine("1)search orders from a certain
customer");
            Console.WriteLine("2)All orders that include a certain
team");
            Console.WriteLine("3)All orders with a certain price");
            Console.WriteLine("4)All orders from a certain city");
            Console.WriteLine("9) exit");
            Console.WriteLine("Please enter your choice: ");
            certainOrdersChoice = int.Parse(Console.ReadLine());
            error = false;
            while (certainOrdersChoice != 1 && certainOrdersChoice
!= 2 && certainOrdersChoice != 3 && certainOrdersChoice != 4 &&
certainOrdersChoice != 9)
            {
                Console.WriteLine("please enter a valid choice
(1,2,3,4 or 9 to exit) ");
                certainOrdersChoice = int.Parse(Console.ReadLine());
                error = false;
            }
        }
        catch
        {
            Console.WriteLine("error must be a number (1,2,3,4 or 9
to exit)");
            Console.WriteLine();
            error = true;
        }
    }
    if (certainOrdersChoice == 1)
    {
        GetCertainOrderByCustomer();
    }
    else if (certainOrdersChoice == 2)

```

```

        {
            GetCertainOrderByClub();
        }
        else if (certainOrdersChoice == 3)
        {
            GetCertainOrderByPriceOfShirt();
        }
        else if (certainOrdersChoice == 4)
        {
            GetCertainOrderByCity();
        }
        Console.WriteLine();
        Console.WriteLine("press enter to continue");
        Console.ReadKey();
        conn.Close();
    }
    static void SelectAllOrders()
    {
        SQLiteConnection conn = CreateConnection();
        Console.WriteLine("all orders:");
        Console.WriteLine();

        SQLiteCommand cmd = new SQLiteCommand("SELECT Orders.OrderID,
        FirstName || ' ' || LastName as 'Full Name', Name as 'Product
        Name', Products.price, city, OrderDate, ClubName, quantity, size FROM customer ,
        Products, Orders , FootballClub, OrderDetails WHERE Products.ProductID =
        OrderDetails.ProductID AND Orders.OrderID = OrderDetails.OrderID AND
        FootballClub.clubID = Products.clubID AND customer.CustomerID =
        Orders.CustomerID GROUP BY OrderDetails.OrderDetailsID", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        for (int i = 0; i < result.FieldCount; i++)
        {
            Console.Write(result.GetName(i) + "| ");
        }
        while (result.Read())
        {
            Console.WriteLine();

            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i].ToString().PadRight(1) + "| ");
            }
        }
        result.Close();
        Console.WriteLine();
        Console.WriteLine();
    }
    static void GetCertainOrderByCustomer()
    {
        SQLiteConnection conn = CreateConnection();
        string space = " ";
    }

```

```

        string search = "";
        Dictionary<string, object> attributes = new Dictionary<string,
object>();
        int rowCount = 0;
        Console.WriteLine("search for a certain customers order by
name(not case sensitive):");
        search = Console.ReadLine();
        SQLiteCommand cmd1 = new SQLiteCommand("SELECT Orders.OrderID,
FirstName || ' ' || LastName as 'Full Name',Name as 'Product
Name',Products.price,city,OrderDate,ClubName,quantity,size FROM customer ,
Products,Orders ,FootballClub,OrderDetails WHERE Products.ProductID =
OrderDetails.ProductID AND Orders.OrderID = OrderDetails.OrderID AND
FootballClub.clubID = Products.clubID AND customer.CustomerID =
Orders.CustomerID AND FirstName || ' ' || LastName like'" + search + "%'
GROUP BY OrderDetails.OrderDetailsID", conn);
        SQLiteDataReader result1 = cmd1.ExecuteReader();
        while (result1.Read())
        {
            for (int i = 0; i < result1.FieldCount; i++)
            {
                //adds the title of the collumn from the table and the
row as the key and then stores the data from that row and collumn as the
value(from the dictionary)
                // this allows me to store the entire table in a
dictionary
                attributes.Add(result1.GetName(i) + rowCount,
result1[i]);
            }
            rowCount++;
        }
        for (int i = 0; i < rowCount; i++)
        {
            Console.Write("Order:" + attributes["OrderID" + i] + space +
attributes["OrderDate" + i]);
            Console.WriteLine();
            for (int j = 0; j < 100; j++)
            {
                Console.Write("-");
            }
            Console.WriteLine();
            Console.WriteLine(attributes["Full Name" + i] + space +
attributes["city" + i]);
            Console.WriteLine();
            Console.WriteLine(attributes["Product Name" + i] + "(" +
attributes["ClubName" + i] + ")" + space + "£" + (attributes["Price" + i]) +
space + "size:" + attributes["size" + i] + space + "quantity:" +
attributes["quantity" + i]);
            Console.WriteLine();
            Console.WriteLine();
            Console.WriteLine();
        }
    }

```

```

    }
    static void GetCertainOrderByClub()
    {
        SQLiteConnection conn = CreateConnection();
        string search = "";
        Console.WriteLine("search for a certain football club's orders
by name(not case sensitive):");
        search = Console.ReadLine();
        SQLiteCommand cmd1 = new SQLiteCommand("SELECT Orders.OrderID,
FirstName || ' ' || LastName as 'Full Name',Name as 'Product
Name',Products.price,city,OrderDate,ClubName,quantity,size FROM customer ,
Products,Orders ,FootballClub,OrderDetails WHERE Products.ProductID =
OrderDetails.ProductID AND Orders.OrderID = OrderDetails.OrderID AND
FootballClub.clubID = Products.clubID AND customer.CustomerID =
Orders.CustomerID AND ClubName like'" + search + "%' GROUP BY
OrderDetails.OrderDetailsID", conn);
        SQLiteDataReader result1 = cmd1.ExecuteReader();
        Dictionary<string, object> attributes = new Dictionary<string,
object>();
        int rowCount = 0;
        string space = " ";
        while (result1.Read())
        {
            for (int i = 0; i < result1.FieldCount; i++)
            {
                attributes.Add(result1.GetName(i) + rowCount,
result1[i]);
            }
            rowCount++;
        }
        Console.WriteLine();
        for (int i = 0; i < rowCount; i++)
        {
            Console.Write("Order:" + attributes["OrderID" + i] + space +
attributes["OrderDate" + i]);
            Console.WriteLine();
            for (int j = 0; j < 100; j++)
            {
                Console.Write("-");
            }
            Console.WriteLine();
            Console.WriteLine(attributes["Full Name" + i] + space +
attributes["city" + i]);
            Console.WriteLine();
            Console.WriteLine(attributes["Product Name" + i] + "(" +
attributes["ClubName" + i] + ")" + space + "£" + (attributes["Price" + i]) +
space + "size:" + attributes["size" + i] + space + "quantity:" +
attributes["quantity" + i]);
            Console.WriteLine();
            Console.WriteLine();
            Console.WriteLine();
        }
    }
}

```

```

    }
}
static void GetCertainOrderByPriceOfShirt()
{
    SQLiteConnection conn = CreateConnection();
    string search = "";
    Console.WriteLine("search for a certain order by price:");
    search = Console.ReadLine();
    SQLiteCommand cmd1 = new SQLiteCommand("SELECT Orders.OrderID,
FirstName || ' ' || LastName as 'Full Name',Name as 'Product
Name',Products.price,city,OrderDate,ClubName,quantity,size FROM customer ,
Products,Orders ,FootballClub,OrderDetails WHERE Products.ProductID =
OrderDetails.ProductID AND Orders.OrderID = OrderDetails.OrderID AND
FootballClub.clubID = Products.clubID AND customer.CustomerID =
Orders.CustomerID AND Price like'" + search + "%' GROUP BY
OrderDetails.OrderDetailsID", conn);
    SQLiteDataReader result1 = cmd1.ExecuteReader();
    Dictionary<string, object> attributes = new Dictionary<string,
object>();
    int rowCount = 0;
    string space = " ";
    while (result1.Read())
    {
        for (int i = 0; i < result1.FieldCount; i++)
        {
            attributes.Add(result1.GetName(i) + rowCount,
result1[i]);
        }
        rowCount++;
    }
    Console.WriteLine();
    for (int i = 0; i < rowCount; i++)
    {
        Console.Write("Order:" + attributes["OrderID" + i] + space +
attributes["OrderDate" + i]);
        Console.WriteLine();
        for (int j = 0; j < 100; j++)
        {
            Console.Write("-");
        }
        Console.WriteLine();
        Console.WriteLine(attributes["Full Name" + i] + space +
attributes["city" + i]);
        Console.WriteLine();
        Console.WriteLine(attributes["Product Name" + i] + "(" +
attributes["ClubName" + i] + ")") + space + "£" + (attributes["Price" + i]) +
space + "size:" + attributes["size" + i] + space + "quantity:" +
attributes["quantity" + i]);
        Console.WriteLine();
        Console.WriteLine();
    }
}

```

```

        Console.WriteLine();
    }
}
static void GetCertainOrderByCity()
{
    SQLiteConnection conn = CreateConnection();
    string search = "";
    Console.WriteLine("search for a certain order by city(not case
sensitive):");
    search = Console.ReadLine();

    SQLiteCommand cmd1 = new SQLiteCommand("SELECT Orders.OrderID,
FirstName || ' ' || LastName as 'Full Name',Name as 'Product
Name',Products.price,city,OrderDate,ClubName,quantity,size FROM customer ,
Products,Orders ,FootballClub,OrderDetails WHERE Products.ProductID =
OrderDetails.ProductID AND Orders.OrderID = OrderDetails.OrderID AND
FootballClub.clubID = Products.clubID AND customer.CustomerID =
Orders.CustomerID AND city like'" + search + "%' GROUP BY
OrderDetails.OrderDetailsID", conn);
    SQLiteDataReader result1 = cmd1.ExecuteReader();
    Dictionary<string, object> attributes = new Dictionary<string,
object>();
    int rowCount = 0;
    string space = " ";
    while (result1.Read())
    {
        for (int i = 0; i < result1.FieldCount; i++)
        {
            attributes.Add(result1.GetName(i) + rowCount,
result1[i]);
        }
        rowCount++;
    }
    Console.WriteLine();
    for (int i = 0; i < rowCount; i++)
    {
        Console.Write("Order:" + attributes["OrderID" + i] + space +
attributes["OrderDate" + i]);
        Console.WriteLine();
        for (int j = 0; j < 100; j++)
        {
            Console.Write("-");
        }
        Console.WriteLine();
        Console.WriteLine(attributes["Full Name" + i] + space +
attributes["city" + i]);
        Console.WriteLine();
        Console.WriteLine(attributes["Product Name" + i] + "(" +
attributes["ClubName" + i] + ")" + space + "£" + (attributes["Price" + i]) +

```

```

space + "size:" + attributes["size" + i] + space + "quantity:" +
attributes["quantity" + i]);
    Console.WriteLine();
    Console.WriteLine();
    Console.WriteLine();

    }
}
static int GetMainMenuChoices()
{
    Console.Clear();
    Console.WriteLine("MAIN MENU");
    Console.WriteLine("which action do you want to complete?");
    Console.WriteLine("-----");
    Console.WriteLine("1)Non-order tables");
    Console.WriteLine("2)Make an order");
    Console.WriteLine("3)order tables");
    Console.WriteLine("4)View all orders");
    Console.WriteLine("5)view graphs and statistics");
    Console.WriteLine("9)exit");

    int ID = 0;
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.Write("Please enter your choice: ");
            ID = int.Parse(Console.ReadLine());
            error = false;
            while (ID != 1 && ID != 2 && ID != 3 && ID != 4 && ID !=
5 && ID != 9)
            {
                Console.WriteLine("please enter a valid choice
(1,2,3,4,5 or 9 to exit) ");
                ID = int.Parse(Console.ReadLine());
                error = false;
            }
        }
        catch
        {
            Console.WriteLine("error must be a number (1,2,3,4,5 or
9 to exit)");

            Console.WriteLine();
            error = true;
        }
    }
    Console.WriteLine();
    return ID;
}

```

```

static int GetSecondMenuChoices()
{
    Console.Clear();
    Console.WriteLine("what action would you like to complete");
    Console.WriteLine("1)view the table");
    Console.WriteLine("2)insert values into the table");
    Console.WriteLine("3)edit the records in the table");
    Console.WriteLine("4)alter the table's columnns");
    Console.WriteLine("9)main menu");
    int ID = 0;
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine();
            Console.Write("Please enter your choice: ");
            ID = int.Parse(Console.ReadLine());
            error = false;
            while (ID != 1 && ID != 2 && ID != 3 && ID != 4 && ID !=
9)
                {
                    Console.WriteLine("please enter a valid choice
(1,2,3,4 or 9 to exit) ");
                    ID = int.Parse(Console.ReadLine());
                    error = false;
                }
        }
        catch
        {
            Console.WriteLine("error must be a number (1,2,3,4 or 9
to exit)");
            Console.WriteLine();
            error = true;
        }
    }
    Console.WriteLine();
    Console.Clear();
    return ID;
}

static void ViewTable(int ID)
{
    SQLiteConnection conn = CreateConnection();
    SQLiteCommand cmd = new SQLiteCommand("SELECT * from " +
tableID[ID], conn);
    SQLiteDataReader result = cmd.ExecuteReader();
    for (int i = 0; i < result.FieldCount; i++)
    {
        Console.Write(result.GetName(i) + "| ");
    }
}

```

```

while (result.Read())
{
    Console.WriteLine();

    for (int i = 0; i < result.FieldCount; i++)
    {
        Console.Write(result[i] + "| ");
    }

    conn.Close();
    Console.WriteLine();
    Console.WriteLine("press enter to return to main menu");
    Console.ReadKey();
}

static void EditTable(int ID)
{
    SQLiteConnection conn = CreateConnection();
    List<string> tableIDString = new List<string>();
    int columnnToChangeInt = 0;
    string newEntry = "";
    int IdToChange = 0;

    SQLiteCommand cmd = new SQLiteCommand("SELECT * from " +
tableID[ID], conn);
    SQLiteDataReader result = cmd.ExecuteReader();
    for (int i = 0; i < result.FieldCount; i++)
    {
        Console.Write(result.GetName(i) + " ");
        tableIDString.Add(result.GetName(i));
    }
    Console.WriteLine();
    Console.WriteLine();
    while (result.Read())
    {
        for (int i = 0; i < result.FieldCount; i++)
        {
            Console.Write(result[i] + " ");
        }
        Console.WriteLine();
    }
    Console.WriteLine();
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("enter which column would you like to
edit (listed below)");
            Console.WriteLine();
            for (int i = 0; i < result.FieldCount; i++)

```

```

        {
            Console.WriteLine((i + 1) + ")" + result.GetName(i) + "
");
        }
        Console.WriteLine("9)exit");
        columnnToChangeInt = int.Parse(Console.ReadLine());
        error = false;
        while (columnnToChangeInt != 1 && columnnToChangeInt !=
2 && columnnToChangeInt != 3 && columnnToChangeInt != 4 &&
columnnToChangeInt != 5 && columnnToChangeInt != 6 && columnnToChangeInt !=
7 && columnnToChangeInt != 9)
        {
            Console.WriteLine("please enter a valid choice
(1,2,3,4,5,6,7 or 9 to exit) ");
            columnnToChangeInt = int.Parse(Console.ReadLine());
            error = false;
        }
    }
    catch
    {
        Console.WriteLine("error must be a number (1,2,3,4,5,6,7
or 9 to exit)");
        Console.WriteLine();
        error = true;
    }
}
if (columnnToChangeInt != 9)
{
    error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("which ID does the data entry have
that you want to change have");
            IdToChange = int.Parse(Console.ReadLine());
            error = false;
        }
        catch
        {
            Console.WriteLine("error must be a number ");
            Console.WriteLine();
            error = true;
        }
    }
}
Console.WriteLine();
Console.WriteLine("what would you like to change this to");
newEntry = Console.ReadLine();

```

```

        SQLiteCommand cmd1 = new SQLiteCommand("UPDATE " +
tableID[ID] + " SET " + tableIDString[columnnToChangeInt - 1] + " = '" +
newEntry + "' WHERE " + result.GetName(0) + " = " + IdToChange, conn);
        cmd1.ExecuteReader();
    }
    Console.WriteLine("press enter to continue");
    Console.ReadKey();

}
static void AlterTable(int ID)
{
    int choice = 0;
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("would you like to do to the " +
tableID[ID] + " table:");
            Console.WriteLine("1)add a column");
            Console.WriteLine("2)remove a column");
            Console.WriteLine("3)rename a column");
            Console.WriteLine("9)exit");
            choice = int.Parse(Console.ReadLine());
            error = false;
            while (choice != 1 && choice != 2 && choice != 3 &&
choice != 9)
            {
                Console.WriteLine("please enter a valid choice (1,2
or 3) ");
                choice = int.Parse(Console.ReadLine());
                error = false;
            }
        }
        catch
        {
            Console.WriteLine("error must be a number (1,2,3)");
            Console.WriteLine();
            error = true;
        }
    }
    switch (choice)
    {
        case 1:
            AddColumn(ID);
            break;
        case 2:
            RemoveColumn(ID);
            break;
        case 3:

```

```

        RenameTable(ID);
        break;
    case 9:
        break;
    default:
        throw new Exception();
    }
}
static void AddColumn(int ID)
{
    SQLiteConnection conn = CreateConnection();
    string columnName = "";
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("what would you like to call the new
column?");

            columnName = Console.ReadLine();
            error = false;
        }
        catch
        {
            Console.WriteLine("error must be a string ");
            Console.WriteLine();
            error = true;
        }
    }
    Console.WriteLine("the data type will be string (VARCHAR)");
    Console.WriteLine("press enter to continue ");
    Console.ReadKey();

    SQLiteCommand cmd = new SQLiteCommand("ALTER TABLE " +
tableID[ID] + " ADD " + columnName + " VARCHAR", conn);
    cmd.ExecuteNonQuery();
}
static void RemoveColumn(int ID)
{
    SQLiteConnection conn = CreateConnection();
    int columnIndex = 0;
    List<string> columnIDString = new List<string>();
    bool error = true;
    bool errorPrint = false;
    while (error == true)
    {
        error = false;
        try
        {

```

```

        Console.WriteLine("which column would you like to
remove");

        SQLiteCommand cmd1 = new SQLiteCommand("SELECT * from "
+ tableID[ID], conn);
        SQLiteDataReader result = cmd1.ExecuteReader();

        for (int i = 0; i < result.FieldCount; i++)
        {
            Console.WriteLine((i + 1) + ") " +
result.GetName(i));
            columnIDString.Add(result.GetName(i));
        }
        columnIndex = int.Parse(Console.ReadLine());
        error = false;
        errorPrint = true;
        for (int i = 0; i < result.FieldCount; i++)
        {
            if (columnIndex == i + 1)
            {
                errorPrint = false;
            }
        }
        while (errorPrint == true)
        {
            Console.WriteLine("please enter a valid number");
            for (int i = 0; i < columnIDString.Count; i++)
            {
                Console.WriteLine((i + 1) + ") " +
columnIDString[i]);
            }
            columnIndex = int.Parse(Console.ReadLine());
            for (int i = 0; i < result.FieldCount; i++)
            {
                if (columnIndex == i + 1)
                {
                    errorPrint = false;
                }
            }
        }
        result.Close();
    }
    catch
    {
        Console.WriteLine("error must be a number");
        Console.WriteLine();
        error = true;
    }
}

```

```

        SQLiteCommand cmd6 = new SQLiteCommand("ALTER TABLE " +
tableID[ID] + " DROP COLUMN " + columnIDString[columnIndex - 1], conn);
        cmd6.ExecuteNonQuery();
        Console.WriteLine("press enter to continue");
        Console.ReadKey();
    }
    static void RenameTable(int ID)
    {
        SQLiteConnection conn = CreateConnection();
        int columnIndex = 0;
        string newColumnName = "";
        List<string> columnIDString = new List<string>();
        bool error = true;
        bool errorPrint = false;
        while (error == true)
        {
            error = false;
            try
            {
                Console.WriteLine("which column would you like to
rename?");
                SQLiteCommand cmd3 = new SQLiteCommand("SELECT * from "
+ tableID[ID], conn);
                SQLiteDataReader result1 = cmd3.ExecuteReader();
                for (int i = 0; i < result1.FieldCount; i++)
                {
                    Console.WriteLine((i + 1) + ") " +
result1.GetName(i));
                    columnIDString.Add(result1.GetName(i));
                }
                columnIndex = int.Parse(Console.ReadLine());
                error = false;
                errorPrint = true;
                for (int i = 0; i < result1.FieldCount; i++)
                {
                    if (columnIndex == i + 1)
                    {
                        errorPrint = false;
                    }
                }
            }
            while (errorPrint == true)
            {
                Console.WriteLine("please enter a valid number");
                for (int i = 0; i < columnIDString.Count; i++)
                {
                    Console.WriteLine((i + 1) + ") " +
columnIDString[i]);
                }
                columnIndex = int.Parse(Console.ReadLine());
                for (int i = 0; i < result1.FieldCount; i++)
                {

```

```

                if (columnIndex == i + 1)
                {
                    errorPrint = false;
                }
            }
        }
    }
    catch
    {
        Console.WriteLine("error must be a number (1,2,3)");
        Console.WriteLine();
        error = true;
    }
}
Console.WriteLine("what would you like to rename it to?");
newColumnName = Console.ReadLine();
SQLiteCommand cmd4 = new SQLiteCommand("ALTER TABLE " +
tableID[ID] + " RENAME COLUMN " + columnIDString[columnIndex - 1] + " to " +
newColumnName, conn);
cmd4.ExecuteNonQuery();
}
static void LastRowOfDataEntered(int ID)
{
    if (ID == 0)
    {
        GetLastRowOfDataEnteredCustomer(ID);
    }
    else if (ID == 1)
    {
        GetLastRowOfDataEnteredOrder(ID);
    }
    else if (ID == 2)
    {
        GetLastRowOfDataEnteredOrderDetails(ID);
    }
    else if (ID == 3)
    {
        GetLastRowOfDataEnteredProduct(ID);
    }
    else if (ID == 4)
    {
        GetLastRowOfDataEnteredFootballClub(ID);
    }
    Console.WriteLine();
}
static void GetLastRowOfDataEnteredCustomer(int ID)
{
    SQLiteConnection conn = CreateConnection();

```

```

        SQLiteCommand cmd = new SQLiteCommand("SELECT * FROM " +
tableID[ID] + " WHERE CustomerID =(SELECT max(customerID) FROM customer)",
conn);

        SQLiteDataReader result = cmd.ExecuteReader();
        while (result.Read())
        {
            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i] + " ");
            }
        }
        result.Close();
        conn.Close();
    } //get last row of data entered unique to each table (below)
    static void GetLastRowOfDataEnteredOrder(int ID)
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd = new SQLiteCommand("SELECT * FROM " +
tableID[ID] + " WHERE OrderID =(SELECT max(OrderID) FROM " + tableID[ID] +
")", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        while (result.Read())
        {
            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i] + " ");
            }
        }
        result.Close();
        conn.Close();
    }
    static void GetLastRowOfDataEnteredOrderDetails(int ID)
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd = new SQLiteCommand("SELECT * FROM " +
tableID[ID] + " WHERE OrderDetailsID =(SELECT max(OrderDetailsID) FROM " +
tableID[ID] + ")", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        while (result.Read())
        {
            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i] + " ");
            }
        }
        result.Close();
        conn.Close();
    }
    static void GetLastRowOfDataEnteredProduct(int ID)
    {
        SQLiteConnection conn = CreateConnection();

```

```

        SQLiteCommand cmd = new SQLiteCommand("SELECT * FROM " +
tableID[ID] + " WHERE ProductID =(SELECT max(ProductID) FROM " + tableID[ID]
+ ")", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        while (result.Read())
        {
            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i] + " ");
            }
        }
        result.Close();
        conn.Close();
    }
    static void GetLastRowOfDataEnteredFootballClub(int ID)
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd = new SQLiteCommand("SELECT * FROM " +
tableID[ID] + " WHERE ClubID =(SELECT max(ClubID) FROM " + tableID[ID] +
")", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        while (result.Read())
        {
            for (int i = 0; i < result.FieldCount; i++)
            {
                Console.Write(result[i] + " ");
            }
        }
        result.Close();
        conn.Close();
    }
    static void InsertIntoTable(int ID)
    {
        string[] dataEntries = new string[8];
        switch (ID)
        {
            case 0:
                //customers
                InsertIntoCustomer(ID);
                break;
            case 1:
                //orders
                InsertIntoOrders(ID);
                break;
            case 2:
                //order details
                InsertIntoOrderDetails(ID);
                break;
            case 3:
                //products

```

```

        InsertIntoProducts(ID);
        break;
    case 4:
        //football club
        InsertIntoFootballClub(ID);
        break;
    case 5:
        //stats table
        InsertIntoStatsTable();
        break;
    default:
        throw new Exception("error incorrect table ID");
    }
    Console.WriteLine("press enter to return to main menu");
    Console.ReadKey();
}
static void InsertIntoCustomer(int ID)
{
    SQLiteConnection conn = CreateConnection();
    string[] dataEntries = new string[8];
    Console.WriteLine("the last row of data entered:");
    LastRowOfDataEntered(ID);
    Console.WriteLine();
    Console.WriteLine("please enter data for each column :
firstname,lastname,address,postal code, country,city (Customer ID will be
automatically incremented)");
    Console.WriteLine("enter first name:");
    dataEntries[0] = Console.ReadLine();
    Console.WriteLine("enter last name:");
    dataEntries[1] = Console.ReadLine();
    Console.WriteLine("enter address:");
    dataEntries[2] = Console.ReadLine();
    Console.WriteLine("enter postage code:");
    dataEntries[3] = Console.ReadLine();
    Console.WriteLine("enter country:");
    dataEntries[4] = Console.ReadLine();
    Console.WriteLine("enter city:");
    dataEntries[5] = Console.ReadLine();
    SQLiteCommand cmd = new SQLiteCommand("INSERT INTO " +
tableID[ID] + " VALUES('" + GetMaxIncrementNum(ID) + "','" + dataEntries[0]
+ "','" + dataEntries[1] + "','" + dataEntries[2] + "','" + dataEntries[3]
+ "','" + dataEntries[4] + "','" + dataEntries[5] + "')", conn);
    cmd.ExecuteNonQuery();
    conn.Close();
}
static void InsertIntoOrders(int ID)
{
    SQLiteConnection conn = CreateConnection();
    string[] dataEntries = new string[8];
    Console.WriteLine("the last row of data entered:");
    LastRowOfDataEntered(ID);

```

```

        Console.WriteLine();
        Console.WriteLine("please enter data for each column : customer
ID (order ID will be automatically incremented and order Date will be
today's date)");
        Console.WriteLine("enter customer ID:");
        dataEntries[0] = Console.ReadLine();
        SQLiteCommand cmd = new SQLiteCommand("INSERT INTO " +
tableID[ID] + " VALUES('" + GetMaxIncrementNum(ID) + "','" + dataEntries[0]
+ "','" + DateTime.Now.ToString().Substring(0, 10) + "')", conn);
        cmd.ExecuteNonQuery();
        Console.WriteLine("you must now enter data into order details to
make sure the database logic still works");
        Console.WriteLine("the last row of data entered:");
        LastRowOfDataEntered(2);
        Console.WriteLine();
        Console.WriteLine("please enter data for each column : Product
ID,quantity (order details ID will be automatically incremented)");
        Console.WriteLine("enter product ID:");
        dataEntries[0] = Console.ReadLine();
        Console.WriteLine("enter quantity:");
        dataEntries[1] = Console.ReadLine();
        SQLiteCommand cmd2 = new SQLiteCommand("INSERT INTO " +
tableID[2] + " VALUES('" + GetMaxIncrementNum(2) + "','" +
GetMaxIncrementNum(2) + "','" + dataEntries[0] + "','" + dataEntries[1] +
"')", conn);
        cmd2.ExecuteNonQuery();
        conn.Close();
    }
    static void InsertIntoOrderDetails(int ID)
    {
        SQLiteConnection conn = CreateConnection();
        string[] dataEntries = new string[8];
        Console.WriteLine("the last row of data entered:");
        LastRowOfDataEntered(ID);
        Console.WriteLine();
        Console.WriteLine("please enter data for each column : customer
ID (order ID will be automatically incremented and order Date will be
today's date)");
        Console.WriteLine("enter customer ID:");
        dataEntries[0] = Console.ReadLine();
        SQLiteCommand cmd3 = new SQLiteCommand("INSERT INTO " +
tableID[ID] + " VALUES('" + GetMaxIncrementNum(ID) + "','" + dataEntries[0]
+ "','" + DateTime.Now.ToString().Substring(0, 10) + "')", conn);
        cmd3.ExecuteNonQuery();
        Console.WriteLine("now you can enter data into order details");
        Console.WriteLine("the last row of data entered:");
        LastRowOfDataEntered(ID);
        Console.WriteLine();
        Console.WriteLine("please enter data for each column : Product
ID,quantity (order details ID will be automatically incremented)");
        Console.WriteLine("enter product ID:");

```

```

        dataEntries[0] = Console.ReadLine();
        Console.WriteLine("enter quantity:");
        dataEntries[1] = Console.ReadLine();
        SQLiteCommand cmd4 = new SQLiteCommand("INSERT INTO " +
tableID[2] + " VALUES('" + GetMaxIncrementNum(2) + "','" +
GetMaxIncrementNum(2) + "','" + dataEntries[0] + "','" + dataEntries[1] +
"')", conn);
        cmd4.ExecuteNonQuery();
        conn.Close();
    }
    static void InsertIntoProducts(int ID)
    {
        SQLiteConnection conn = CreateConnection();
        string[] dataEntries = new string[8];
        Console.WriteLine("the last row of data entered:");
        LastRowOfDataEntered(ID);
        Console.WriteLine();
        Console.WriteLine("please enter data for each column :
Name,stock,description,size,price,clubID (product ID will be automatically
incremented)");
        Console.WriteLine("enter name:");
        dataEntries[0] = Console.ReadLine();
        Console.WriteLine("enter stock:");
        dataEntries[1] = Console.ReadLine();
        Console.WriteLine("enter description:");
        dataEntries[2] = Console.ReadLine();
        Console.WriteLine("enter size:");
        dataEntries[3] = Console.ReadLine();
        Console.WriteLine("enter price:");
        dataEntries[4] = Console.ReadLine();
        Console.WriteLine("enter clubID:");
        dataEntries[5] = Console.ReadLine();
        SQLiteCommand cmd5 = new SQLiteCommand("INSERT INTO " +
tableID[ID] + " VALUES('" + GetMaxIncrementNum(ID) + "','" + dataEntries[0]
+ "','" + dataEntries[1] + "','" + dataEntries[2] + "','" + dataEntries[3]
+ "','" + dataEntries[4] + "','" + dataEntries[5] + "')", conn);
        cmd5.ExecuteNonQuery();
        conn.Close();
    }
    static void InsertIntoFootballClub(int ID)
    {
        SQLiteConnection conn = CreateConnection();
        string[] dataEntries = new string[8];
        Console.WriteLine("the last row of data entered:");
        LastRowOfDataEntered(ID);
        Console.WriteLine();
        Console.WriteLine("please enter data for each column :Club Name
(Club ID will be automatically incremented)");
        Console.WriteLine("enter club name:");
        dataEntries[0] = Console.ReadLine();

```

```

        SQLiteCommand cmd6 = new SQLiteCommand("INSERT INTO " +
tableID[ID] + " VALUES('" + GetMaxIncrementNum(ID) + "',' + dataEntries[0]
+ "')", conn);
        cmd6.ExecuteNonQuery();
        conn.Close();
    }
    static void InsertIntoStatsTable()
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd7 = new SQLiteCommand("INSERT INTO
stats(totalProduct, totalCustomer, totalOrder,DateOfEntry) VALUES((SELECT
COUNT(*) FROM Products), (SELECT COUNT(*) FROM customer), (SELECT COUNT(*)
FROM Orders),' + DateTime.Now.ToString().Substring(0, 10) + "')", conn);
        cmd7.ExecuteNonQuery();
        conn.Close();
    }
    static int GetMaxIncrementNum(int ID)
    {
        if (ID == 0)
        {
            return GetMaxIncrementNumCustomerID();
        }
        else if (ID == 1)
        {
            return GetMaxIncrementNumOrderID();
        }
        else if (ID == 2)
        {
            return GetMaxIncrementNumOrderDetailsID();
        }
        else if (ID == 3)
        {
            return GetMaxIncrementNumProductID();
        }
        else if (ID == 4)
        {
            return GetMaxIncrementNumclubID();
        }
        else
        {
            throw new Exception("error incorrect table ID");
        }
    }
    static int GetMaxIncrementNumCustomerID()
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd = new SQLiteCommand("SELECT max(customerID)
FROM customer", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
    }
}

```

```

        cmd = conn.CreateCommand();
        int myReader = 0;
        while (result.Read())
        {
            myReader = result.GetInt16(0);
        }
        result.Close();
        conn.Close();
        myReader++;
        return myReader;
    }
    static int GetMaxIncrementNumOrderID()
    {
        SQLiteConnection conn = CreateConnection();

        SQLiteCommand cmd = new SQLiteCommand("SELECT max(OrderID) FROM
Orders", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        cmd = conn.CreateCommand();
        int myReader = 0;
        while (result.Read())
        {
            myReader = result.GetInt16(0);
        }
        result.Close();
        conn.Close();
        myReader++;
        return myReader;
    }
    static int GetMaxIncrementNumOrderDetailsID()
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd = new SQLiteCommand("SELECT
max(OrderDetailsID) FROM OrderDetails", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        cmd = conn.CreateCommand();
        int myReader = 0;
        while (result.Read())
        {
            myReader = result.GetInt16(0);
        }
        result.Close();
        conn.Close();
        myReader++;
        return myReader;
    }
    static int GetMaxIncrementNumProductID()
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd = new SQLiteCommand("SELECT max(ProductID)
FROM Products", conn);

```

```

        SQLiteDataReader result = cmd.ExecuteReader();
        cmd = conn.CreateCommand();
        int myReader = 0;
        while (result.Read())
        {
            myReader = result.GetInt16(0);
        }
        result.Close();
        conn.Close();
        myReader++;
        return myReader;
    }
    static int GetMaxIncrementNumclubID()
    {
        SQLiteConnection conn = CreateConnection();
        SQLiteCommand cmd = new SQLiteCommand("SELECT max(clubID) FROM
FootballClub", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        int myReader = 0;
        while (result.Read())
        {
            myReader = result.GetInt16(0);
        }
        result.Close();
        conn.Close();
        myReader++;
        return myReader;
    }
    static void GraphChoice(int ID)
    {
        ID = 0;
        bool error = true;
        while (error == true)
        {
            error = false;
            try
            {
                Console.WriteLine("which graph would you like to view
");
                Console.WriteLine("1) The sum of profit from each day");
                Console.WriteLine("2) The price of the shirt that was
sold each day");
                Console.WriteLine("3) The total products each day");
                Console.WriteLine("4) The total amount of customers each
day");
                Console.WriteLine("5) The total amount of orders each
day");

                Console.WriteLine("9) exit to main menu");
                ID = int.Parse(Console.ReadLine());
                error = false;
            }
            catch { }
        }
    }
}

```

```

        while (ID != 1 && ID != 2 && ID != 3 && ID != 4 && ID !=
5 && ID != 9)
        {
            Console.WriteLine("please enter a valid choice
(1,2,3,4,5,9) ");

            ID = int.Parse(Console.ReadLine());
            error = false;
        }
    }
    catch
    {
        Console.WriteLine("error must be a number (1,2,3,4,5)");
        Console.WriteLine();
        error = true;
    }
}
if (ID != 9)
{
    double[,] BothAxis = GetAxis(ID);
    double[] x = new double[BothAxis.GetUpperBound(0) + 1];
    double[] y = new double[BothAxis.GetUpperBound(0) + 1];
    for (int i = 0; i < BothAxis.GetUpperBound(0) + 1; i++)
    {
        y[i] = BothAxis[i, 0];
        x[i] = BothAxis[i, 1];
    }
    Console.Clear();

    PrintGraph(x, y, y.Length, ID);

    //// test data 1
    //double[] testX = new double[24] { 16, 17, 18, 19, 20, 21,
13, 14, 5, 7, 20, 9, 1, 24, 1, 2, 3, 4, 5, 6, 7, 8, 9, 15 };
    //double[] testY = new double[24] { 16, 17, 18, 19, 20, 21,
5, 4, 6, 20, 7, 1, 1, 24, 1, 2, 3, 4, 5, 6, 7, 8, 9, 15 };

    //// test data 2
    //double[] testX = new double[12] { 16, 17, 20, 30, 40, 50,
60, 70, 100, 200, 500, 1000 };
    //double[] testY = new double[12] { 16, 17, 20, 30, 40, 50,
60, 70, 100, 200, 500, 1000 };

    //PrintGraph(testX, testY, testY.Length, ID);
}
}
static string GetMonthOfData()
{
    int month = 0;

```

```

string monthString = "0";

bool error = true;
while (error == true)
{
    error = false;
    try
    {
        Console.WriteLine("which month would you like to
view?");

        Console.WriteLine("1)January");
        Console.WriteLine("2)February");
        Console.WriteLine("3)March");
        Console.WriteLine("4)April");
        Console.WriteLine("5)May");
        Console.WriteLine("6)June");
        Console.WriteLine("7)July");
        Console.WriteLine("8)August");
        Console.WriteLine("9)September");
        Console.WriteLine("10)October");
        Console.WriteLine("11)November");
        Console.WriteLine("12)December");
        month = int.Parse(Console.ReadLine());
        error = false;
        while (month != 1 && month != 2 && month != 3 && month
!= 4 && month != 5 && month != 6 && month != 7 && month != 8 && month != 9
&& month != 10 && month != 11 && month != 12)
        {
            Console.WriteLine("please enter a valid choice
(1,2,3,4,5,6,7,8,9,10,11 or 12 ");
            month = int.Parse(Console.ReadLine());
            error = false;
        }
    }
    catch
    {
        Console.WriteLine("error must be a number
(1,2,3,4,5,6,7,8,9,10,11 or 12");
        Console.WriteLine();
        error = true;
    }
}
if (month.ToString().Length > 1)
{
    monthString = month.ToString();
}
else
{
    monthString += month.ToString();
}
return monthString;

```

```

    }
    static double[,] GetAxis(int ID)
    {
        List<string> getData = new List<string>();
        List<double> Xaxis = new List<double>();
        List<double> Yaxis = new List<double>();
        string monthString = GetMonthOfData();
        if (ID == 1)
        {
            return GetSumOfProfitForGraph(getData, Xaxis, Yaxis,
monthString);
        }

        else if (ID == 2)
        {
            return GetPriceOfShirtForGraph(getData, Xaxis, Yaxis,
monthString);
        }
        else if (ID == 3)
        {
            return GetTotalProductsForGraph(getData, Xaxis, Yaxis,
monthString);
        }
        else if (ID == 4)
        {
            return GetTotalAmountOfCustomersForGraph(getData, Xaxis,
Yaxis, monthString);
        }
        else if (ID == 5)
        {
            return GetTotalAmountOfOrdersForGraph(getData, Xaxis, Yaxis,
monthString);
        }
        else
        {
            throw new Exception("graph does not exist");
        }
    }
    static double[,] GetSumOfProfitForGraph(List<string> getData,
List<double> Xaxis, List<double> Yaxis, string monthString)
    {
        SQLiteConnection conn = CreateConnection();
        //1) The sum of profit from each day
        SQLiteCommand cmd = new SQLiteCommand("SELECT
sum(price),OrderDate FROM customer,Orders,OrderDetails ,Products WHERE
customer.CustomerID = Orders.CustomerID AND Orders.OrderID =
OrderDetails.OrderID AND OrderDetails.ProductID = Products.ProductID AND
substr(OrderDate,4,2)='" + monthString + "' GROUP BY OrderDate", conn);
        SQLiteDataReader result = cmd.ExecuteReader();
        while (result.Read())
    }

```

```

    {
        for (int i = 0; i < result.FieldCount; i++)
        {
            getData.Add(result[i].ToString());
        }
    }
    conn.Close();
    for (int i = 0; i < getData.Count; i++)
    {
        if (i % 2 == 0)
        {
            Yaxis.Add(double.Parse(getData[i]));
        }
        else
        {
            Xaxis.Add(double.Parse(getData[i].Substring(0, 2)));
        }
    }
    double[,] dataOut = new double[Yaxis.Count, 2];
    for (int i = 0; i < Yaxis.Count; i++)
    {
        dataOut[i, 0] = Yaxis[i];
        dataOut[i, 1] = Xaxis[i];
    }
    return dataOut;
}

static double[,] GetPriceOfShirtForGraph(List<string> getData,
List<double> Xaxis, List<double> Yaxis, string monthString)
{
    SQLiteConnection conn = CreateConnection();
    //2) The price of each shirt sold each day
    SQLiteCommand cmd = new SQLiteCommand("SELECT price, OrderDate
from Orders ,OrderDetails, Products WHERE substr(OrderDate,4,2)='" +
monthString + "' AND Products.ProductID = OrderDetails.ProductID AND
Orders.OrderID = OrderDetails.OrderID", conn);

    SQLiteDataReader result = cmd.ExecuteReader();
    while (result.Read())
    {
        for (int i = 0; i < result.FieldCount; i++)
        {
            getData.Add(result[i].ToString());
        }
    }
    conn.Close();
    for (int i = 0; i < getData.Count; i++)
    {
        if (i % 2 == 0)
        {

```

```

        Yaxis.Add(double.Parse(getData[i]));
    }
    else
    {

        Xaxis.Add(double.Parse(getData[i].Substring(0, 2)));
    }
}

double[,] dataOut = new double[Yaxis.Count, 2];
for (int i = 0; i < Yaxis.Count; i++)
{
    dataOut[i, 0] = Yaxis[i];
    dataOut[i, 1] = Xaxis[i];
}
return dataOut;
}

static double[,] GetTotalProductsForGraph(List<string> getData,
List<double> Xaxis, List<double> Yaxis, string monthString)
{
    //3) The total products each day
    SQLiteConnection conn = CreateConnection();
    SQLiteCommand cmd = new SQLiteCommand("SELECT totalProduct,
DateOfEntry from stats WHERE substr(DateOfEntry,4,2)='" + monthString + "'",
conn);

    SQLiteDataReader result = cmd.ExecuteReader();
    while (result.Read())
    {
        for (int i = 0; i < result.FieldCount; i++)
        {
            getData.Add(result[i].ToString());
        }
    }
    conn.Close();
    for (int i = 0; i < getData.Count; i++)
    {
        if (i % 2 == 0)
        {
            Yaxis.Add(double.Parse(getData[i]));
        }
        else
        {

            Xaxis.Add(double.Parse(getData[i].Substring(0, 2)));
        }
    }
    double[,] dataOut = new double[Yaxis.Count, 2];
    for (int i = 0; i < Yaxis.Count; i++)
    {
        dataOut[i, 0] = Yaxis[i];
        dataOut[i, 1] = Xaxis[i];
    }
}

```

```

    }
    return dataOut;
}
static double[,] GetTotalAmountOfCustomersForGraph(List<string>
getData, List<double> Xaxis, List<double> Yaxis, string monthString)
{
    SQLiteConnection conn = CreateConnection();
    //4) The total amount of customers each day
    SQLiteCommand cmd = new SQLiteCommand("SELECT totalCustomer,
DateOfEntry from stats WHERE substr(DateOfEntry,4,2)='" + monthString + "'",
conn);

    SQLiteDataReader result = cmd.ExecuteReader();
    while (result.Read())
    {
        for (int i = 0; i < result.FieldCount; i++)
        {
            getData.Add(result[i].ToString());
        }
    }
    conn.Close();
    for (int i = 0; i < getData.Count; i++)
    {
        if (i % 2 == 0)
        {
            Yaxis.Add(double.Parse(getData[i]));
        }
        else
        {
            Xaxis.Add(double.Parse(getData[i].Substring(0, 2)));
        }
    }
    double[,] dataOut = new double[Yaxis.Count, 2];
    for (int i = 0; i < Yaxis.Count; i++)
    {
        dataOut[i, 0] = Yaxis[i];
        dataOut[i, 1] = Xaxis[i];
    }
    return dataOut;
}
static double[,] GetTotalAmountOfOrdersForGraph(List<string>
getData, List<double> Xaxis, List<double> Yaxis, string monthString)
{
    SQLiteConnection conn = CreateConnection();
    //5) The total amount of orders each day
    SQLiteCommand cmd = new SQLiteCommand("SELECT totalOrder,
DateOfEntry from stats WHERE substr(DateOfEntry,4,2)='" + monthString + "'",
conn);

    SQLiteDataReader result = cmd.ExecuteReader();
    while (result.Read())
    {

```

```

        for (int i = 0; i < result.FieldCount; i++)
        {
            getData.Add(result[i].ToString());
        }
    }
    conn.Close();
    for (int i = 0; i < getData.Count; i++)
    {
        if (i % 2 == 0)
        {
            Yaxis.Add(double.Parse(getData[i]));
        }
        else
        {
            Xaxis.Add(double.Parse(getData[i].Substring(0, 2)));
        }
    }
    double[,] dataOut = new double[Yaxis.Count, 2];
    for (int i = 0; i < Yaxis.Count; i++)
    {
        dataOut[i, 0] = Yaxis[i];
        dataOut[i, 1] = Xaxis[i];
    }
    return dataOut;
}

static void PrintGraph(double[] x, double[] y, int n, int ID)
{
    // prints graph to console
    Console.WindowHeight = 40;
    //printing out the axis
    Console.CursorTop = graphDistFromTop;
    for (int i = 0; i < (graphHight); i++)
    {
        if (i == 0)
        {
            Console.CursorLeft = graphDistFromSide - 2;
            Console.Write("20");
        }
        else if (i == 5)
        {
            Console.CursorLeft = graphDistFromSide - 2;
            Console.Write("15");
        }
        else if (i == 10)
        {
            Console.CursorLeft = graphDistFromSide - 2;
            Console.Write("10");
        }
        else if (i == graphHight - 5)

```

```

        {
            Console.CursorLeft = graphDistFromSide - 2;
            Console.Write("50");
        }
        else if (i == graphHight - 1)
        {
            Console.CursorLeft = graphDistFromSide - 1;
            Console.Write("0");
        }
        Console.CursorLeft = graphDistFromSide;
        Console.WriteLine("|");
    }
    Console.CursorLeft = graphDistFromSide;
    // originally 40 (left point)
    for (int i = 0; i < GraphWidth; i++)
    {

        Console.Write("-");
    }
    Console.SetCursorPosition(0, 0);
    PrintDataset(n, x, y);
    //printing out the data x and y at the top of the screen (for
    reference with graph)
    PrintAxis(ID);
    //checks to see if co-ordinate is a duplicate and changes it's
    colour based on how many times it is repeated
    CheckIfDataIsAduplicateAndPrint(ID, n, x, y);
    // prints a key of colours to show how many duplicates of a data
    point there is
    DataDupliacteKeyPrint();
    //prints the type of error that happened to the graph
    PrintErrorWithGraph(x, y);
}
static void PrintDataset(int n, double[] x, double[] y)
{
    for (int i = 0; i < n; i++)
    {
        Console.Write("x=" + x[i]);
    }
    Console.WriteLine();
    for (int i = 0; i < n; i++)
    {
        Console.Write("y=" + y[i]);
    }
}
static void CheckIfDataIsAduplicateAndPrint(int ID, int n, double[]
x, double[] y)
{
    List<string> DupeString = new List<string>();
    int dupeCount = 0;
    for (int i = 0; i < n; i++)

```

```

    {
        dupeCount = 0;
        DupeString.Add((x[i]).ToString() + (y[i] / 10).ToString());
        for (int j = 0; j < DupeString.Count; j++)
        {
            if ((x[i]).ToString() + (y[i] / 10).ToString() ==
DupeString[j] && i != j)
            {
                dupeCount++;
            }
        }
        // if the chosen graph type is 1 or 2 then the y value will
be divided by 10
        if (ID == 1 || ID == 2)
        {
            if (dupeCount == 0)
            {
                Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i] / 10));
                Console.Write("x");
            }
            if (dupeCount == 1)
            {
                Console.ForegroundColor = ConsoleColor.Red;
                Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i] / 10));
                Console.Write("x");
            }
            if (dupeCount == 2)
            {
                Console.ForegroundColor = ConsoleColor.Yellow;
                Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i] / 10));
                Console.Write("x");
            }
            if (dupeCount == 3)
            {
                Console.ForegroundColor = ConsoleColor.Blue;
                Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i] / 10));
                Console.Write("x");
            }
            if (dupeCount == 4)
            {
                Console.ForegroundColor = ConsoleColor.Magenta;
                Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i] / 10));
                Console.Write("x");
            }
            if (dupeCount > 4)

```

```

        {
            Console.ForegroundColor = ConsoleColor.Green;
            Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i] / 10));
            Console.Write("x");
        }
    }
    // if the chosen graph type is 3 4 or 5 then the y value
    will not be divided by 10
    else if (ID == 3 || ID == 4 || ID == 5)
    {
        if (dupeCount == 0)
        {
            Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i]));
            Console.Write("x");
        }
        if (dupeCount == 1)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i]));
            Console.Write("x");
        }
        if (dupeCount == 2)
        {
            Console.ForegroundColor = ConsoleColor.Yellow;
            Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i]));
            Console.Write("x");
        }
        if (dupeCount == 3)
        {
            Console.ForegroundColor = ConsoleColor.Blue;
            Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i]));
            Console.Write("x");
        }
        if (dupeCount == 4)
        {
            Console.ForegroundColor = ConsoleColor.Magenta;
            Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i]));
            Console.Write("x");
        }
        if (dupeCount > 4)
        {
            Console.ForegroundColor = ConsoleColor.Green;
            Console.SetCursorPosition(graphDistFromSide +
(int)(x[i]), graphmidpointY - (int)(y[i]));

```

```

        Console.WriteLine("x");
    }
}
Console.ForegroundColor = ConsoleColor.White;
}
}
static void PrintErrorWithGraph(double[] x, double[] y)
{
    double pmmcVal = Pmcc(x, y);
    double LeastSquareRegressionVal= LeastSquareRegression(x, y);

    if (x.Length == 0 || y.Length == 0)
    {
        Console.SetCursorPosition(0, 30);
        Console.WriteLine("the graph is empty because there is no
data for this month yet");
        Console.WriteLine("press enter to continue");
        Console.ReadKey();
    }
    else
    {
        if (pmmcVal.ToString() == "NaN" || LeastSquareRegressionVal
== -1)
        {
            Console.SetCursorPosition(0, 30);
            Console.WriteLine("error there is not enough data to
show correlation between points or predict the future or the data is
invalid");

            Console.WriteLine("press enter to continue");
            Console.ReadKey();
        }
        else
        {
            Console.SetCursorPosition(70, 27);
            Console.WriteLine("this graph has The Product Moment
Correlation Coefficient");
            Console.SetCursorPosition(70, 28);
            Console.WriteLine("of " + Math.Round(pmmcVal, 2));
            Console.SetCursorPosition(0, 27);
            if (LeastSquareRegressionVal == -1)
            {
                Console.WriteLine("error Least Square Regression
could not be calculated");
            }
        }
    }
}
static void DataDupliacteKeyPrint()
{
    Console.SetCursorPosition(100, 0);
    Console.WriteLine("x = no same data (white)");
}

```

```

        Console.SetCursorPosition(100, 1);
        Console.ForegroundColor = ConsoleColor.Red;
        Console.Write("x");
        Console.ForegroundColor = ConsoleColor.White;
        Console.Write("= 1 duplicate of data (red)");
        Console.SetCursorPosition(100, 2);
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.Write("x");
        Console.ForegroundColor = ConsoleColor.White;
        Console.Write("= 2 duplicate data (Yellow)");
        Console.SetCursorPosition(100, 3);
        Console.ForegroundColor = ConsoleColor.Blue;
        Console.Write("x");
        Console.ForegroundColor = ConsoleColor.White;
        Console.Write("= 3 duplicate data (Blue)");
        Console.SetCursorPosition(100, 4);
        Console.ForegroundColor = ConsoleColor.Magenta;
        Console.Write("x");
        Console.ForegroundColor = ConsoleColor.White;
        Console.Write("= 4 duplicate data (Purple)");
        Console.SetCursorPosition(100, 5);
        Console.ForegroundColor = ConsoleColor.Green;
        Console.Write("x");
        Console.ForegroundColor = ConsoleColor.White;
        Console.Write("= more than 4 duplicates of data (green)");
    }
    static void PrintAxis(int ID)
    {
        // prints the graphs axis differntly depedning on which data is
        being used
        Console.SetCursorPosition(0, 10);
        if (ID == 1 || ID == 2)
        {
            Console.WriteLine("Y axis");
            Console.WriteLine();
            Console.WriteLine("P|");
            Console.WriteLine(" |");
            Console.WriteLine("R|");
            Console.WriteLine(" |");
            Console.WriteLine("I|");
            Console.WriteLine(" |");
            Console.WriteLine("C|");
            Console.WriteLine(" |");
            Console.WriteLine("E|");
            Console.SetCursorPosition(20, 26);
            Console.WriteLine("X axis Date(day)");
            Console.SetCursorPosition(19, 27);
            Console.WriteLine("-----");
        }
        else if (ID == 3 || ID == 4 || ID == 5)
        {

```

```

        Console.WriteLine("Y axis");
        Console.WriteLine();
        Console.WriteLine("T|");
        Console.WriteLine(" |");
        Console.WriteLine("O|");
        Console.WriteLine(" |");
        Console.WriteLine("T|");
        Console.WriteLine(" |");
        Console.WriteLine("A|");
        Console.WriteLine(" |");
        Console.WriteLine("L|");
        Console.SetCursorPosition(20, 26);
        Console.WriteLine("X axis Date(day)");
        Console.SetCursorPosition(19, 27);
        Console.WriteLine("-----");
    }
}

static double Pmcc(double[] x, double[] y)
{
    double n = x.Length;
    double xMean = 0;
    double yMean = 0;
    double xMinusXMean = 0;
    double xMinusXMeanTotal = 0;
    double xMinusXMeanTotalSquared = 0;
    double yMinusYMean = 0;
    double yMinusYMeanTotal = 0;
    double yMinusYMeanTotalSquared = 0;

    double numerator = 0;
    double denominator = 0;
    double pmccID = 0;

    for (int i = 0; i < n; i++)
    {
        xMean += x[i];
    }
    xMean /= n;
    for (int i = 0; i < n; i++)
    {
        yMean += y[i];
    }
    yMean = yMean / n;

    for (int i = 0; i < n; i++)
    {
        xMinusXMean = x[i] - xMean;
        xMinusXMeanTotal += xMinusXMean;
        xMinusXMeanTotalSquared += Math.Pow(xMinusXMean, 2);

        yMinusYMean = y[i] - yMean;

```

```

        yMinusYMeanTotal += yMinusYMean;
        yMinusYMeanTotalSquared += Math.Pow(yMinusYMean, 2);

        numerator += xMinusXMean * yMinusYMean;
    }

    denominator = xMinusXMeanTotalSquared * yMinusYMeanTotalSquared;
    pmccID = numerator / Math.Sqrt(denominator);
    return pmccID;
}

static double LeastSquareRegression(double[] x, double[] y)
{
    double n = y.Length;
    double sumx = 0;
    double sumy = 0;
    double sumxy = 0;
    double c = 0;
    double yTarget = 0;
    double xPredict = 0;
    double sumXSquared = 0;
    double m = 0;
    for (int i = 0; i < n; i++)
    {
        sumxy += x[i] * y[i];

        sumx += x[i];
        sumy += y[i];

        sumXSquared += x[i] * x[i];
    }

    m = (n * sumxy - (sumx * sumy))
        / (n * sumXSquared - (sumx * sumx));

    c = (sumy - m * sumx) / n;
    if (!(m.ToString() == "NaN" || c.ToString() == "NaN"))
    {
        Console.SetCursorPosition(0, 30);

        Console.WriteLine("y = mx + c:");
        Console.WriteLine("y = " + Math.Round(m, 3) + " x + " +
Math.Round(c, 3));

        bool error = true;
        while (error == true)
        {
            error = false;
            try
            {
                Console.WriteLine("whats your target y");
                yTarget = double.Parse(Console.ReadLine());
            }

```

```

        error = false;
    }
    catch
    {
        Console.WriteLine("error must be a number exit");
        Console.WriteLine();
        error = true;
    }
}
xPredict = (yTarget - c) / m;
Console.WriteLine("you will reach your target when x =" +
Math.Round(xPredict, 3) + " --> thats " + Math.Round((xPredict - x[x.Length
- 1]), 3) + " days more than the latest x value (which is the date: " +
(x[x.Length - 1] + "th)"));
Console.WriteLine("press enter to exit to main menu");
Console.ReadKey();
return xPredict;
}
return -1;
}
static void CustomerTable(int ID)
{
    //sends the user to differnt actions for the customer table
    switch (GetSecondMenuChoices())
    {
        case 1:
            ViewTable(ID);
            break;
        case 2:
            InsertIntoTable(ID);
            break;
        case 3:
            EditTable(ID);
            break;
        case 4:
            AlterTable(ID);
            break;
        case 9:
            break;
    }
}
static void OrderTable(int ID)
{
    //sends the user to differnt actions for the Order Table
    switch (GetSecondMenuChoices())
    {
        case 1:
            ViewTable(ID);
            break;
        case 2:

```

```

        InsertIntoTable(ID);
        break;
    case 3:
        EditTable(ID);
        break;
    case 4:
        AlterTable(ID);
        break;
    case 9:

        break;
    }
}
static void OrderDetailsTable(int ID)
{
    //sends the user to differnt actions for the OrderDetailsTable
    switch (GetSecondMenuChoices())
    {
        case 1:
            ViewTable(ID);
            break;
        case 2:
            InsertIntoTable(ID);
            break;
        case 3:
            EditTable(ID);
            break;
        case 4:
            AlterTable(ID);
            break;
        case 9:

            break;
    }
}
static void ProductTable(int ID)
{
    //sends the user to differnt actions for the ProductTable
    switch (GetSecondMenuChoices())
    {
        case 1:
            ViewTable(ID);
            break;
        case 2:
            InsertIntoTable(ID);
            break;
        case 3:
            EditTable(ID);
            break;
        case 4:
            AlterTable(ID);

```

```

        break;
    case 9:

        break;
    }
}
static void FootballClubTable(int ID)
{
    //sends the user to differnt actions for the FootballClubTable
    switch (GetSecondMenuChoices())
    {
        case 1:
            ViewTable(ID);
            break;
        case 2:
            InsertIntoTable(ID);
            break;
        case 3:
            EditTable(ID);
            break;
        case 4:
            AlterTable(ID);
            break;
        case 9:

            break;
    }
}
static void StatsTable(int ID)
{
    //sends the user to differnt actions for the StatsTable
    int choice = 0;
    bool error = true;
    while (error == true)
    {
        error = false;
        try
        {
            Console.WriteLine("what action would you like to
complete");

            Console.WriteLine("1)view the stats table");
            Console.WriteLine("2)add to stats table(will enter the
stats from today's date)");
            Console.WriteLine("9) exit");
            choice = int.Parse(Console.ReadLine());
            error = false;
            while (choice != 1 && choice != 2 && choice != 9)
            {
                Console.WriteLine("please enter a valid choice (1,2
or 9 to exit) ");
                choice = int.Parse(Console.ReadLine());
            }
        }
        catch { error = true; }
    }
}

```

```

        error = false;
    }
}
catch
{
    Console.WriteLine("error must be a number (1,2 or 9 to
exit)");
    Console.WriteLine();
    error = true;
}
}
switch (choice)
{
    case 1:
        ViewTable(ID);
        break;
    case 2:
        InsertIntoTable(ID);
        break;
    case 9:
        break;
}
}
static SQLiteConnection CreateConnection()
{
    SQLiteConnection sqlite_conn;
    // Create a new database connection:
    sqlite_conn = new SQLiteConnection("Data Source=Football shirt
database.db;Version=3;New=True;Compress=True;");
    // Open the connection:
    sqlite_conn.Open();
    return sqlite_conn;
}
}
}

```